

RobotWorld: Benchmarking Multimodal Agents for Robot Use Across Diverse Tasks and Embodiments

Zhiqin Yang*, Chenxin Li*,†, Xiaomeng Hu*,†, Yibin Liu, Weidong Huang, Jiankai Sun, Haitao Li, Zijian Wu, Yuzhi Huang, Fanding Huang, Hanwen Sun, Jiashun Liu, Jingqi Tong, Mingxin Huang, Shaoli Hu, Shijue Huang, Tianyi Bai, Xinyuan Wang, Yunlong Lin, Zhengyang Tang, Zhexin Zhang, Zhuo Chen, Xierui Song, Juntao Dai, Boyuan Chen, Jiaming Ji, Fangneng Zhan, Mengkang Hu, Wei Xue, Yonggang Zhang, Han Hu, Tsung-Yi Ho, Yike Guo



*Equal contribution †Project leads [Project Page](#) [Code](#)



General-purpose agents increasingly write code, use tools, and complete complex digital tasks, raising the question of how far these capabilities carry into the physical world. To investigate this, we introduce **RobotWorld**, a challenging simulation testbed for *robot use*: turning instructions and observations into physical task execution through robot interfaces. Its 84 tasks span manipulation, mobile manipulation, locomotion, driving, and aerial control, with explicit interaction budgets and executable success checks. By analysing task outcomes alongside execution traces, we identify both the capabilities that transfer and the gaps that prevent reliable completion. Furthermore, we find that current agents can construct sophisticated perception and control workflows, including image segmentation, camera calibration, spatial estimation, and dynamics-based computation. These capabilities, however, do not consistently compose into successful behaviour: agents lose task-relevant object states despite reaching commanded poses, fail to correct ineffective actions, recover too late, or mistake unfinished tasks for completion. This uneven transfer also differs across models: Astra succeeds more often on spatial and constrained-contact goals, whereas Opus 5.5 succeeds more often on continuous-balance and timed-interaction goals. By linking these outcomes to execution behaviour, RobotWorld provides both a rigorous proving ground and an empirical account of the remaining capability gaps, thereby establishing concrete targets for training and designing more reliable physical-world agents.

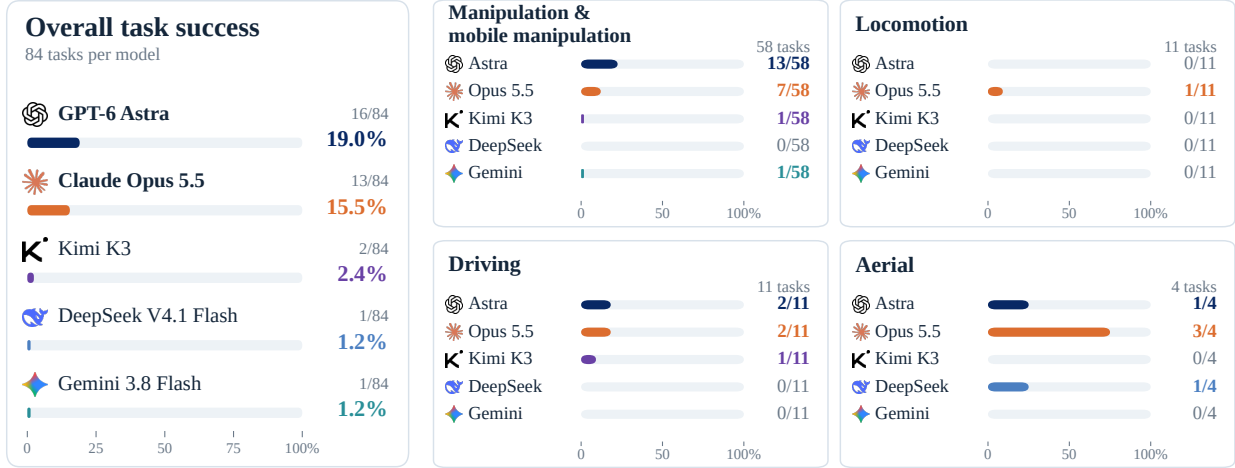


Figure 1 RobotWorld leaderboard. Left: overall success across 84 tasks. Right: success in four displayed groups (Manipulation and Mobile manipulation are pooled), with counts shown beside each bar. All panels use the same 0–100% success scale. Each model has one retained outcome per task under the evaluation budget rules.

1 Introduction

General-purpose agents are becoming useful actors in the digital world. They develop software, operate browsers, and carry out multi-step knowledge-work tasks through frontier harnesses and systems such as Operator and Claude Cowork (OpenAI, 2025; Anthropic, 2026b). Their capabilities extend beyond producing answers: they inspect an environment, use tools, observe the consequences, and revise their actions. Robotics presents a natural next question: **how far can these capabilities take agents into the physical world?** Recent studies of frontier models, including GPT-6 Astra, have made robot control an increasingly concrete possibility (Anthropic, 2026a; Galbot Team et al., 2026). Community experiments now include robot-arm control and simulated dexterous apple grasping, while real-time robot rollouts also expose the gap between an impressive demonstration and practical execution speed (Chooi, 2026; Huang, 2026; Duan, 2026). The challenge is to establish which capabilities already transfer, which remain missing, and what should improve next.

These experiments have also prompted debate about what progress in robot use should mean. Malik questions whether pick-and-place demonstrations establish competence in dexterity and dynamics; Isola and Fu emphasise the potential of agents that use controllers and robotics tools (Malik, 2026; Isola, 2026; Fu, 2026). This motivates a broader empirical question: across which physical demands can a general-purpose agent act reliably, with what control support, and where does its feedback loop fail? Physical interaction tests how perception, reasoning, and program construction work together. An agent may estimate a grasp from images, write a segmentation program, fit camera geometry, or calculate a controller from approximate dynamics. Each is useful only insofar as it supports successful action. Describing a grasp does not secure the object; reaching a target pose does not establish insertion; recovering balance for an instant does not satisfy sustained stability. Across different robot bodies, the agent must interpret its observations, anticipate action consequences, correct errors, and judge completion. We use *robot use* to describe this ability to turn instructions and feedback into task execution through robot interfaces.

Recent work is turning general-purpose models into agents that execute robot tasks through programs, tools, and feedback. EmbodiedSWE studies coding agents that construct executable solutions for long-horizon dexterous tasks (Shen et al., 2026). PyRUA-Lean investigates how programmatic action composition and selective observation improve execution efficiency (Si et al., 2026), while RPG uses simulation practice and failure diagnosis to refine reusable robot skills (Wang et al., 2026a). These advances make systematic evaluation increasingly important: across which robot bodies and physical demands do current agents act reliably, and where does execution break down? We study this question across manipulation, locomotion, driving, and flight, connecting task outcomes to how agents combine perception, computation, and feedback. Our aim is to turn promising demonstrations into measurable capability gaps and concrete targets for future training and agent design.

We introduce RobotWorld, a simulation benchmark of robot use across diverse tasks and embodiments. It brings together 84 task specifications with documented robot interfaces, task budgets, and executable success checks (see Figure 2). Multimodal agents interact through a shared execution harness, with each task’s observations and controller assistance specified. Recorded actions, tool interactions, and outcomes support analysis beyond aggregate scores. Sections 3–5 describe the environments, benchmark construction, and evaluation protocol; our central empirical question is what these executions reveal about agents’ physical capabilities.

Our trajectory analysis reveals both substantial capabilities and persistent gaps. Agents construct perception and control workflows involving colour segmentation, camera-geometry fitting, spatial estimation, and numerical dynamics calculations. They also use feedback to revise motions and

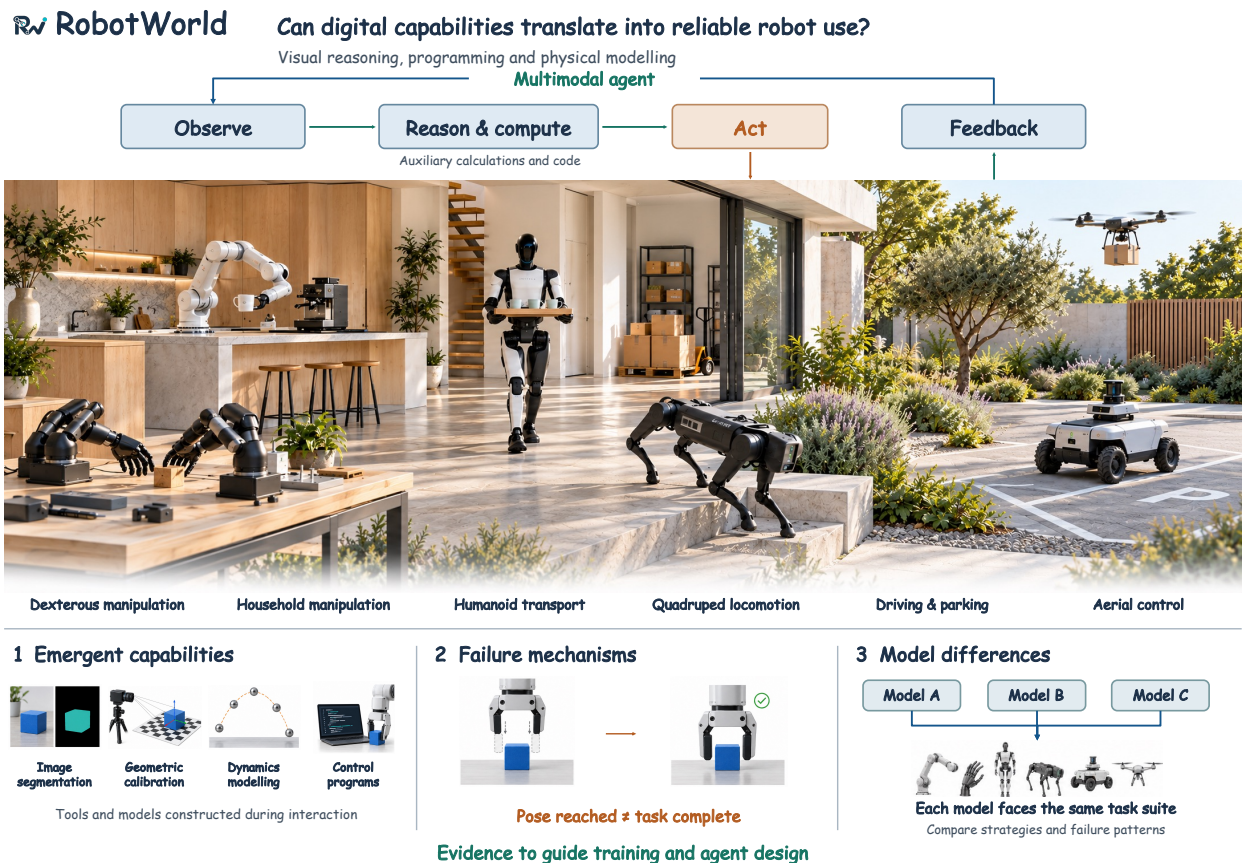


Figure 2 Overview of RobotWorld. RobotWorld evaluates whether general-purpose multimodal models can turn digital capabilities into reliable robot use. Agents interpret instructions and observations, perform auxiliary computation, and act through a feedback loop across diverse robots and tasks. Trajectory analysis examines emerging capabilities—including image segmentation, geometric calibration, dynamics modelling and control program generation—alongside failure mechanisms and differences between models on shared tasks. These findings inform training and agent design. The scene illustrates task families evaluated in separate simulation environments.

recover from execution errors. Yet these local capabilities do not consistently compose into task completion: a robot may reach the requested pose while losing the object, remain in navigation without reaching the next subgoal, regain stability too late, or stop correcting an unfinished task because it believes the goal is already met. These failures locate weaknesses in state interpretation, action revision, temporal coordination, and completion assessment.

The models also exhibit different task and interaction profiles. Across the task groups analysed in Section 6.6, Astra succeeds more often on spatial and constrained-contact goals, whereas Opus 5.5 succeeds more often on continuous-balance and timed-interaction goals. Among eight tasks solved by both, Astra uses fewer tool calls in seven but fewer control steps in only six, showing that interaction cost and physical execution cost need not agree. Together with the observed workflows, these patterns suggest different ways of composing visual-spatial reasoning, explicit computation, and feedback control. We treat this as an exploratory explanation of the model profiles. By connecting outcomes to execution mechanisms, RobotWorld aims to make evaluation useful for directing progress towards more capable physical-world agents.

Table 1 Control interfaces and task coverage (✓: included; ✗: not reported in the cited protocols). Direct actions are model-emitted commands or action values; code control executes generated programs. Mixed calls alternate these interfaces within one episode. Balance denotes tasks with an explicit robot-stability objective. Optional RobotWorld code control is included.

Evaluation	Control interfaces			Beyond manipulation		
	Direct actions	Code control	Mixed calls	Navigation	Balance	Driving / flight
CaP-X (Fu et al., 2026)	✗	✓	✗	✓	✗	✗
EmbodiedBench (Yang et al., 2025)	✓	✗	✗	✓	✗	✗
VLABench (Zhang et al., 2024)	✓	✓	✗	✗	✗	✗
EmbodiedEval (Cheng et al., 2025)	✓	✗	✗	✓	✗	✗
Embodied Agent Interface (Li et al., 2024b)	✓	✗	✗	✓	✗	✗
EmbodiedSWE-Bench (Shen et al., 2026)	✗	✓	✗	✓	✗	✗
RobotWorld (ours)	✓	✓	✓	✓	✓	✓

Our contributions are twofold:

1. **A challenging, rigorous proving ground for robot use.** RobotWorld gives the community a systematic way to test whether general-purpose agents can turn digital capabilities into successful physical action. Its 84 tasks from 20 source projects span diverse embodiments and control demands, with explicit interfaces, budgets, and executable success checks. Substantial remaining headroom makes it a testbed for developing and measuring the next generation of physical-world agents.
2. **An empirical capability profile and directions for improvement.** Through execution traces and cross-model comparisons, we reveal how current agents combine perception, computation, and control, where reliable task completion breaks down, and how strengths and weaknesses differ across models. These findings identify concrete priorities for future training and agent design, including grounding perception and computation in action, recovering from execution errors, coordinating actions over time, and recognising task completion.

2 Related Work

Foundation Models for Robotics. Vision-language-action (VLA) models such as RT-2, OpenVLA and π_0 generate actions from visual observations and language instructions (Brohan et al., 2023; Kim et al., 2024; Black et al., 2024); $\pi_{0.5}$ further studies open-world generalisation through heterogeneous co-training (Black et al., 2025). World-action models (WAMs) connect action generation with future-observation prediction. Recent work examines complementary aspects of this coupling: SelfWAM models action-conditioned visual consequences (Pan et al., 2026), OpenWAM systematically studies world-action pretraining and information flow (Wang et al., 2026b), and Rolling-WAM distributes joint video-action denoising across replanning cycles to improve responsiveness (Zhou et al., 2026). These directions study closely related capabilities: selecting actions from sensory inputs and modelling how actions change the world. Robot-use agents can express both through reasoning, programs and feedback. Our trajectories include direct visual estimates used to choose motions, as well as geometric and dynamics calculations used to anticipate motion and derive control commands. RobotWorld examines how these capabilities are combined during execution and whether they produce reliable task completion across diverse physical demands.

Agents for Robot Use. Agents use reasoning, programs, and tools to connect instructions to physical action. Code as Policies and VoxPoser generate executable control logic and spatial representations for robot motion (Liang et al., 2022; Huang et al., 2023), while Show-Harness exposes semantic robot actions to VLMs (Chen et al., 2026b). EmbodiedSWE studies coding agents that construct executable solutions for dexterous and mobile manipulation (Shen et al., 2026). Recent work develops how such agents use execution feedback: PyRUA-Lean composes robot primitives into Python cells with conditional checks, local retries, and selective observation (Si et al., 2026); ASPIRE accumulates validated program repairs into reusable skills (Lu et al., 2026); and RPG uses simulation practice and failure diagnosis to revise a shared skill library and system prompt (Wang et al., 2026a). These approaches develop complementary ways to organise robot execution, from decisions within an episode to skills retained across tasks. RLE-Bench examines the broader use of coding agents for robotics research and engineering, including policy learning, perception, mechanical design, and interactive control (Ma et al., 2026). Our focus is the breadth and reliability of robot use: how agents combine available capabilities during execution, and which mechanisms support or obstruct task completion.

Benchmarks for Embodied Agents. CaP-X evaluates coding agents for manipulation under different primitive abstractions and feedback settings, with extensions to mobile manipulation (Fu et al., 2026). EmbodiedBench evaluates visual agents across high-level planning and lower-level navigation and manipulation, with capability-oriented task subsets (Yang et al., 2025). VLABench examines language-conditioned manipulation and long-horizon reasoning, evaluating both action policies and VLM-based workflows (Zhang et al., 2024); EmbodiedEval spans navigation, object and social interaction, and embodied question answering (Cheng et al., 2025). At the decision-module level, Embodied Agent Interface separately evaluates goal interpretation, subgoal decomposition, action sequencing, and transition modelling (Li et al., 2024b). These agent evaluations build on a wider ecosystem of robot-learning task suites, including RL Bench, CALVIN, LIBERO, BEHAVIOR-1K, and RoboCasa365 (James et al., 2020; Mees et al., 2022; Liu et al., 2023; Li et al., 2024a; Nasiriany et al., 2026). RobotWorld brings together diverse embodiments and physical demands, including spatial manipulation, sustained stabilisation, and interaction with moving targets. Alongside task outcomes, we analyse feedback use, recovery, completion assessment, and interaction cost to characterise model strengths and failure mechanisms. Table 1 compares control interfaces and coverage beyond manipulation. Direct commands and generated programs are distinct entry points: RobotWorld can interleave both within one episode, while retaining the declared native controllers.

3 RobotWorld Environment

RobotWorld provides a common interaction protocol for robot use across heterogeneous simulators and embodiments. Given an instruction, an agent inspects observations, performs auxiliary computation, and issues robot commands. Execution returns observations and feedback for the next decision. The overall architecture is shown in Figure 3. It connects observations, auxiliary computation, robot tools, and evaluation while preserving each source system’s physics, action semantics, and controller assistance.

3.1 Task Definition and Interaction Loop

A task specifies both a physical goal and the conditions under which the agent must achieve it. We represent task i as

$$\mathcal{T}_i = (\mathcal{E}_i, \rho_i, g_i, O_i, A_i, C_i, B_i, V_i), \quad (1)$$

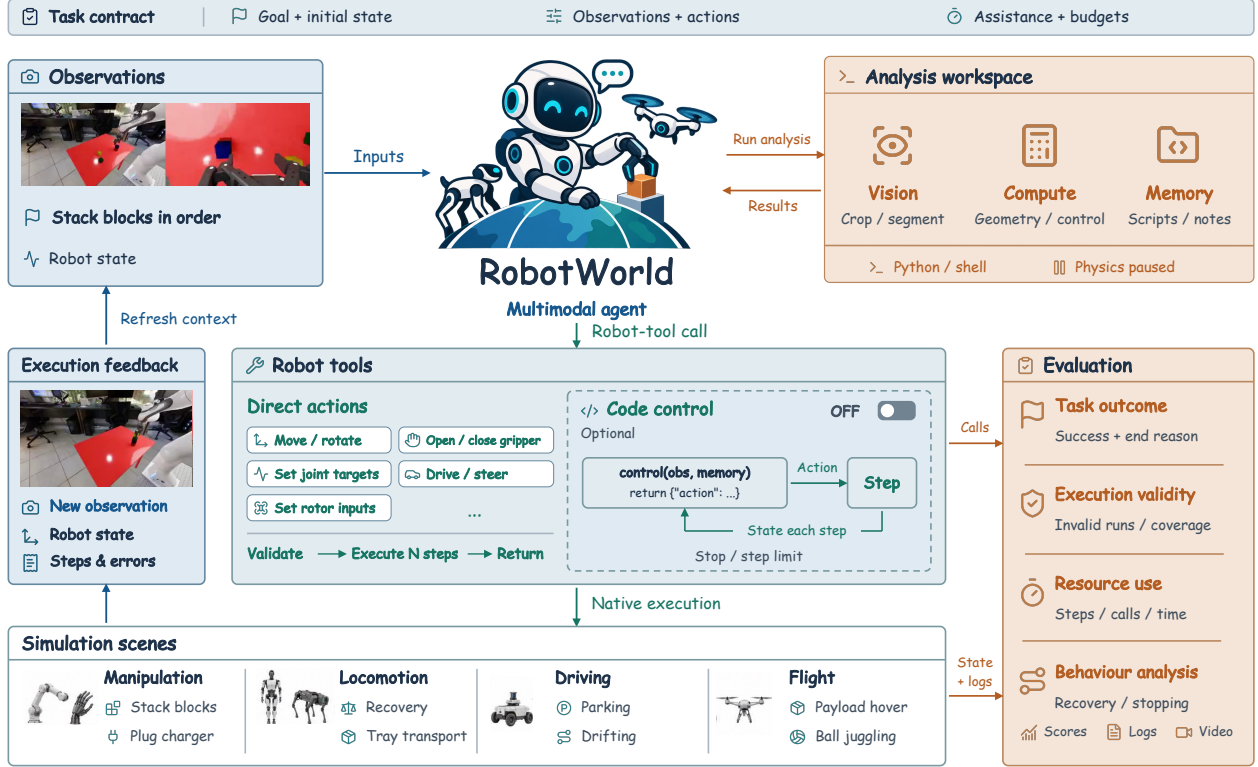


Figure 3 The RobotWorld framework. Under a task-specific observation, action, and budget contract, a multimodal agent alternates between analysis and robot execution, using returned observations and execution feedback to revise its actions. Direct actions execute bounded command segments; optional code control, disabled by default, runs generated programs with feedback at each control step. The analysis workspace supports perception, computation, and memory while physics is paused. Evaluation records task outcomes, execution validity, resource use, and behaviour across manipulation, locomotion, driving, and flight.

where $\mathcal{E}_i$ is the environment, ρ_i defines initial conditions, and g_i is the goal. The observation interface O_i , action interface A_i , and controller assistance C_i define the agent’s access to the robot. Budgets B_i bound interaction, and evaluator V_i assesses the executed trajectory. Together, these elements form a task contract: they expose differences in sensing and control that a shared tool protocol alone would otherwise conceal.

At interaction k , the agent selects a response using the goal, interface documentation D_i , and permitted history h_k :

$$z_k \sim \pi_\theta(\cdot \mid g_i, D_i, h_k). \quad (2)$$

The response may request robot execution, invoke workspace computation, or produce text. Model parameters remain fixed during an episode; observations, conversation history, and permitted workspace contents evolve as the agent acts. A frontier harness manages model conversations and tools, while separate environment processes execute robot requests and return feedback. As shown in Figure 4, a turn proceeds from observation and optional computation to a bounded execution request and returned feedback. Task evaluation follows a separate state-and-event path.

3.2 Observations and Analysis Workspace

Permitted observations. The task instruction and interface documentation accompany camera images, available robot state, and execution feedback. Observation channels are task-specific: an

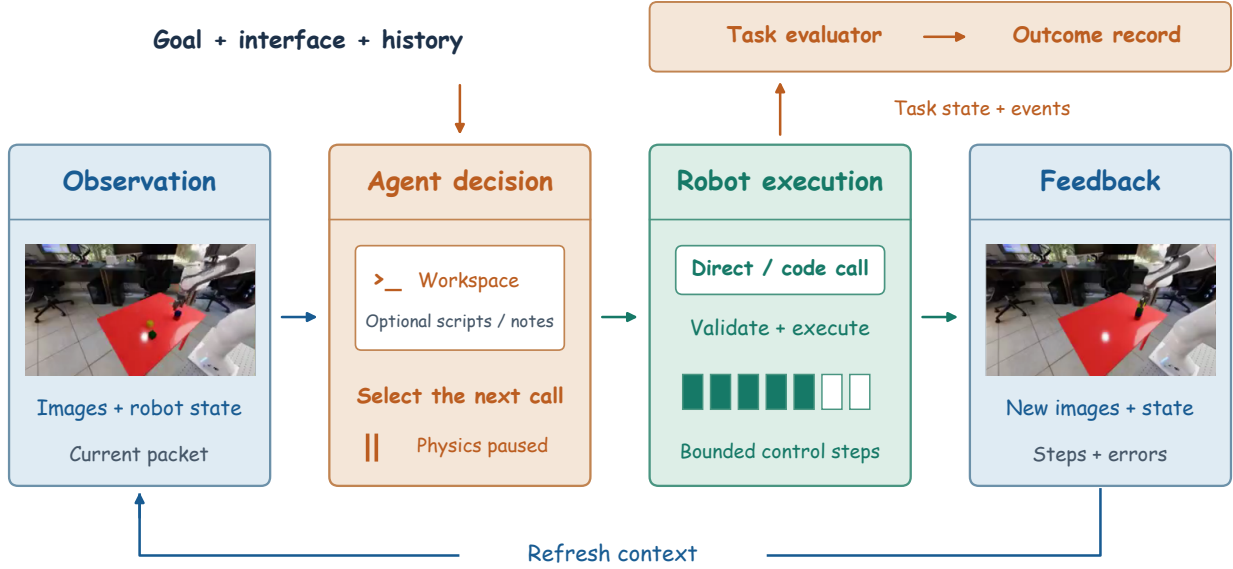


Figure 4 Environment interaction loop. Observations and task context inform the agent’s next robot call, optionally supported by workspace computation. Native execution validates the request and advances physics within its step limit; returned observations, completed steps, and errors inform the next decision. Physics pauses during reasoning and offline computation. The evaluator checks task state and events independently. Camera views illustrate observations from a stacking episode, rather than the endpoints of a single call; the execution strip is schematic.

interface may expose joint positions or object measurements in addition to images. Evaluator-only state and cameras used solely to review trajectories remain outside the agent interface. Consequently, the information used to judge an outcome can be richer than the information available to achieve it.

Auxiliary computation. Where enabled, an isolated workspace supports image inspection, geometric estimation, numerical calculations, and the development of control programs. Scripts and notes can retain intermediate results within an episode. Workspace tools operate on supplied observations and permitted files; they neither access simulator internals nor advance physics. Workspace access and code control are configured independently: analysing an image does not require permission to run a program inside the control loop.

Observation history. Timestamped packets distinguish newly returned observations from earlier evidence. Analysis and text-only continuation retain the latest packet without implying that the scene has changed. Action records and workspace notes support later decisions, subject to the configured history window. The experimental setup specifies the retained history and tool access used for model comparison.

3.3 Robot Actions and Execution Feedback

Robot execution converts the agent’s request into a bounded segment of physical interaction. Direct actions are always available; optional code control adds a feedback loop within a call. As shown in Figure 5, both modes return control to the agent, which can revise its next request using the resulting observations.

Direct actions. A robot-tool call contains action values or targets together with a step count. For example, `move_joints(...)` specifies joint targets. The environment validates the request, executes it through the native controller, and returns permitted observations and execution feedback. This feedback includes completed steps and reported errors, allowing a rejected command to be distinguished from a command that executed but failed to achieve its intended effect.

Code control. Where supported and enabled, an additional tool runs an agent-generated feedback program (Liang et al., 2022). At each control step, the program reads permitted observations, updates local memory, and selects an action without another model call. Execution is bounded by a step limit and may end earlier on a stop condition, error, or episode termination. If the episode continues, the agent can revise the program or return to direct actions. Thus, code control changes the frequency at which a generated program receives feedback, while the agent still replans between tool calls.

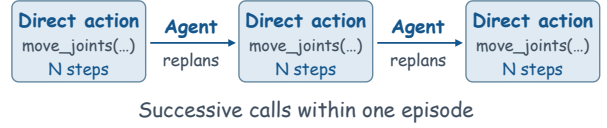
Control semantics and assistance. Interface documentation specifies units, coordinate frames, bounds, and absolute or incremental commands. Both execution modes retain these semantics and the task’s native controllers. Motion generation, such as cuRobo (Sundaralingam et al., 2023) in RoboDojo, and low-level servos are distinguished from learned task or balance policies. These forms of assistance affect the control problem presented to the agent and are part of the task contract; a common protocol does not make all action spaces or control demands equivalent.

3.4 Episode Execution and Records

Initialisation and time. Each episode begins from the prescribed task initialisation. Physics advances during robot execution and pauses while the agent reasons or computes offline. Let n_k denote the cumulative number of native control steps. A request that completes d_k steps gives: $n_{k+1} = n_k + d_k$, with $d_k = 0$ for workspace computation or text. A control step may contain several physics substeps; the native control interval determines simulated duration. Control steps, tool interactions, and wall-clock time therefore measure different aspects of resource use.

Call and episode boundaries. Completing a command segment or returning a stop signal from a program ends that call. Native termination and configured episode limits determine whether further interaction is possible. Neither a completed call nor an agent’s textual completion claim

(a) Code control disabled



(b) Code control enabled

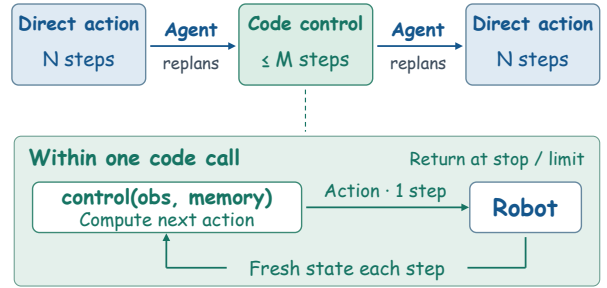


Figure 5 Control is chosen per call. Direct actions and code segments can alternate within one episode. The agent replans between calls; within a code segment, the program updates actions from fresh state at each step, up to its limit of M steps.

establishes task success. The evaluator assesses the relevant states and events during execution, including temporal or history-dependent conditions when required by the task.

Outcome records. Episode records retain observations, requested and completed actions, checker outcomes, and available errors or termination reasons. These records connect an outcome to the physical execution that produced it and support trajectory and video review. They also allow resource consumption to be examined separately from success. Section 4 specifies what the task checkers require; the experimental setup defines the model configurations, resource limits, and aggregation used for the reported comparisons.

4 RobotWorld Benchmark

The environment defines how agents interact with robots. The benchmark defines what they must accomplish and what evidence establishes completion. RobotWorld comprises 84 tasks from 20 source projects, spanning manipulation, mobile manipulation, locomotion, driving, and aerial control. Its construction brings these tasks under explicit instructions, interaction conditions, and executable success criteria, enabling outcomes to be interpreted in terms of the physical demands agents encounter.

4.1 Task Scope and Selection

Task selection targets complementary forms of robot use: establishing object relations, operating articulated objects, coordinating multiple stages, maintaining stability, and acting on moving targets. Variants contribute distinct tasks when their goals or physical constraints change, such as entry direction, terrain, timing, or coordination. Aliases and alternative control modes do not increase the task count.

Domains and task types. Figure 6 summarises the suite at two levels. The five primary domains contain 38 manipulation tasks, 20 mobile manipulation tasks, 11 locomotion tasks, 11 driving tasks, and four aerial tasks. Manipulation uses fixed-base arm or hand interfaces; mobile manipulation uses the mobile base and arm interfaces of RoboCasa and BEHAVIOR-1K. Locomotion covers legged motion and whole-body balance or coordination. Within each domain, a primary type describes the task’s main objective, such as fitting and insertion, balance and recovery, or precision parking. Each task contributes once to this composition. These labels characterise the required activity rather than the frequency of low-level actions in a particular model’s trajectory. Together, manipulation and mobile manipulation account for 69.0% of the suite, so the aggregate should be interpreted alongside results for the other domains.

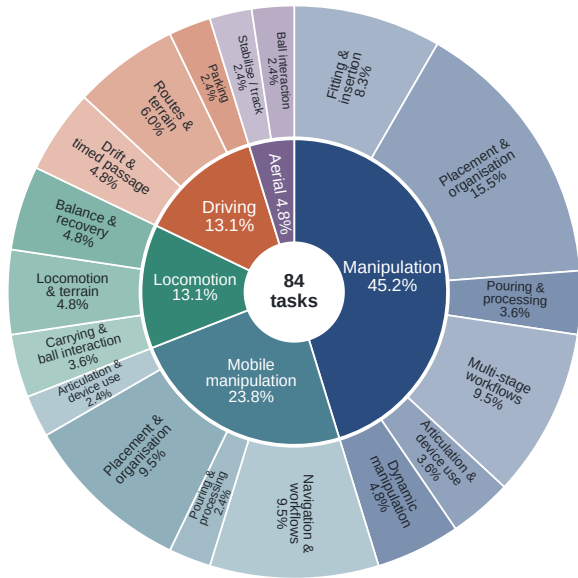


Figure 6 Task composition by domain and primary task type. The inner ring shows five domains and the outer ring their primary task types. Each task is counted once. Percentages use all 84 tasks, not action frequencies.

Shared control requirements. Tasks also share control requirements across domains. *Spatial goals* require particular positions, orientations, destinations, or paths. *Constrained contact* covers interactions such as insertion, hanging, pouring, and articulated-object operation, beyond unconstrained pick-and-place. *Multiple subgoals* requires completing several objects or stages while preserving progress; a routine approach–grasp–lift sequence alone does not qualify. *Balance and tracking* requires sustained regulation of posture, support, or a changing reference. *Timed interaction* requires coordination with an independently moving target or constraint, as in conveyor picking, interception, or moving-gate passage.

These requirements overlap: a task can involve spatial positioning, constrained contact, and several subgoals. Primary types describe what the agent is asked to do; requirement labels describe the challenges involved. Neither constitutes a model performance score or a decomposition into independent abilities.

4.2 Task Construction and Adaptation

Manipulation and mobile manipulation tasks draw on BEHAVIOR-1K (Li et al., 2024a), RoboCasa (Nasiriany et al., 2024, 2026), RoboLab (Yang et al., 2026a), RoboDojo (Chen et al., 2026a), and Bench2Dex (Yang et al., 2026b), alongside additional control tasks. Driving and aerial environments include WheeledLab (Han et al., 2025), OmniDrones (Xu et al., 2023), and VolleyBots (Xu et al., 2025; Ji et al., 2025). Appendix A attributes all 20 source projects to their papers or software releases.

Each benchmark task combines source simulation assets with an instruction, an initialisation procedure, an interaction contract, and a success checker. We distinguish the origin of the objective from changes to the interface or scenario. An inherited goal can be exposed through a modified robot interface, and a new goal can use an existing scene and controller. Treating these as separate dimensions makes the benchmark’s additions explicit.

Objectives and checkers. When a source defines a suitable goal and executable success condition, the benchmark retains them. For example, selected Bench2Dex tasks use source scene anchors and native event and stability checks. In environments designed around a continuing reward, a reward signal alone may not establish completion of an instruction. Selected locomotion and aerial tasks therefore receive explicit completion objectives, such as reaching a destination after disturbances or stabilising a payload within specified bounds. Surviving to the horizon is insufficient unless it satisfies the stated objective.

Interfaces and scenarios. Environment adapters expose permitted observations, documented commands, controller assistance, and bounded execution through the interaction protocol in Section 3. Scenario changes specify the conditions in which those interfaces are used. Seven RobotWorld driving scenarios, for example, introduce constraints including narrow supports, restricted parking spaces, and a moving gate. Scene geometry and disturbances determine the problem faced by the agent; entry direction, final alignment, or valid gate passage determine whether it has been solved.

Task specification. Source identity, initialisation, observation and action interfaces, assistance, budgets, and checker definitions jointly identify an evaluation configuration. Changes to the objective are treated as RobotWorld task definitions rather than implied changes to the source benchmark’s results. Native rewards or auxiliary scores remain distinct from completion under the declared goal. Task-level details and checker thresholds are retained with the benchmark specification.

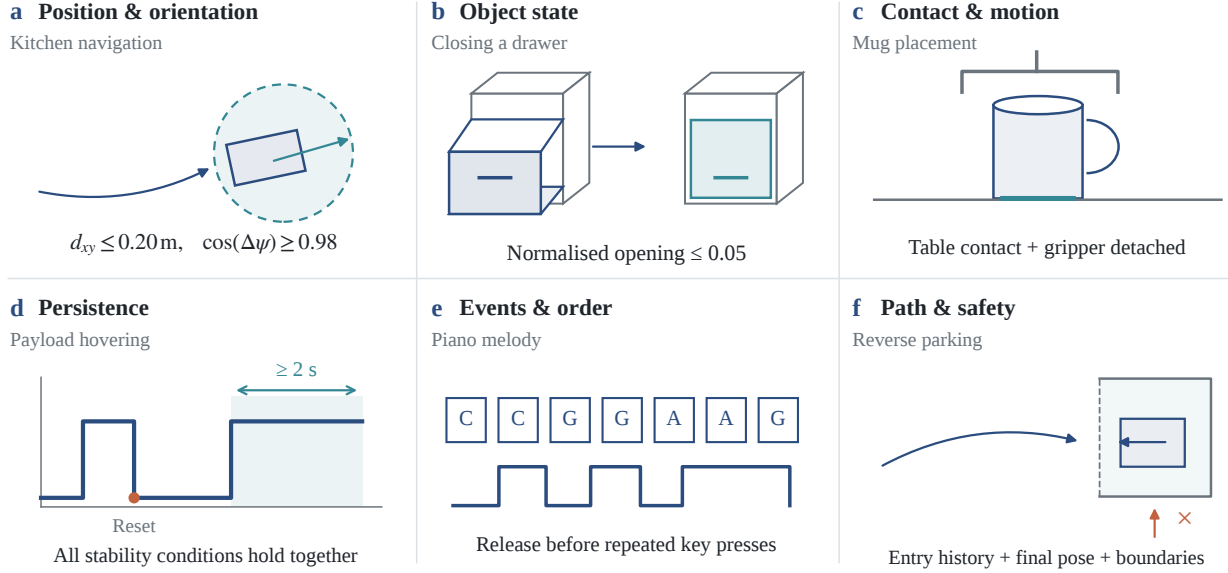


Figure 7 What establishes task completion. Six families of success evidence, illustrated schematically. (a) Kitchen navigation checks position and heading. (b) Drawer closure checks normalised joint opening. (c) Mug placement requires support and release in addition to its spatial goal. (d) Payload hovering requires all stability bounds to hold together for at least the final two seconds; a broken continuous hold resets its duration. (e) Repeated piano notes require release before re-pressing. (f) Reverse parking combines entry history, final pose, and boundary constraints. Thresholds are specific to the illustrated tasks; these are checker components, not universal rules for all 84 tasks.

4.3 Success Criteria

Success is determined from task-relevant simulator states and events in the executed trajectory. Checkers assess physical outcomes rather than agreement with a reference action sequence, allowing different strategies to succeed when they satisfy the same conditions. Figure 7 illustrates six overlapping families of evidence. A task may combine several families; each illustration shows a component of a full task checker.

Position and orientation. Geometric predicates test distances, alignment, insertion depth, containment, or relative placement. Kitchen navigation requires both the target position and heading. An apparent alignment in the image plane need not satisfy a three-dimensional geometric goal.

Object state. Articulation and semantic checks recognise conditions such as a closed drawer, an activated appliance, or frozen food. Object identities, counts, and quantified relations may also matter: placing exactly three qualifying fruits on a plate differs from placing at least three.

Contact and motion. Support, release, or velocity constraints distinguish completed placement from a transient or still-held configuration. The white-mug task, for example, checks table contact and gripper detachment as well as placement near the centre. These conditions are explicit when required; geometric containment alone does not certify physical contact.

Persistence. Hold conditions require simultaneous satisfaction over a duration, whereas fixed-window conditions constrain an entire specified interval. A violation resets a continuous hold; a violation within a fixed evaluation window cannot be erased by subsequent recovery. These checks

Table 2 Representative task coverage by embodiment (✓: represented; ✗: no selected example). Labels overlap and describe task requirements, not model success.

Embodiment	Goal requirements			Dynamic control	
	Spatial goals	Constrained contact	Multiple subgoals	Balance / tracking	Timed interaction
Fixed-base arms	✓	✓	✓	✓	✓
Dexterous / bimanual	✓	✓	✓	✓	✗
Mobile manipulators	✓	✓	✓	✗	✗
Humanoids / bipeds	✗	✓	✓	✓	✓
Quadrupeds	✗	✗	✗	✓	✗
Wheeled / wheel-legged	✓	✗	✓	✓	✓
Aerial robots	✗	✗	✗	✓	✓

operate at the evaluator’s sampling rate. An arrival goal receives no additional dwell requirement unless its checker specifies one.

Events and order. Counters and state machines recognise distinct ball hits, repeated recoveries, or ordered key presses. Repeated piano notes require a release event before the next press. Temporal ordering is imposed only when the goal requires it: a prescribed final stack order constrains spatial relations without necessarily prescribing the order of picks.

Path and safety. History-dependent conditions check route checkpoints, entry direction, or forbidden boundary crossings. A valid final parking pose cannot compensate for a recorded path violation when the task requires reverse entry and boundary clearance.

These criteria connect an instruction to observable completion. A favourable final image, a finished tool call, and the agent’s own success claim can each be insufficient. Which evidence is needed follows from the task definition, independently of the strategy used to attempt it.

4.4 Validation and Coverage

Checker and execution evidence. Validation distinguishes whether a checker implements its specification from whether the goal can be reached through legal actions. Positive, negative, and boundary fixtures test scoring logic, including missing evidence and failure precedence in the RobotWorld state-checking framework. Scene probes, state and event replay, and trajectory or video inspection provide complementary evidence about physical execution. A constructed positive state can test a predicate but does not demonstrate a physically achievable solution.

Feasibility and limits. A successful action trajectory establishes reachability under its tested initial conditions and interface. Reference controllers may use additional state, with that access distinguished from agent observations, while acting through the declared control interface. Reference coverage is incomplete; checker validation does not establish a solution for every task or robustness across initial conditions. The scope of a success claim is further limited by the physical relations represented in its checker and the rate at which they are sampled.

Coverage across embodiments. Table 2 maps representative requirements across seven embodiment groups. A marked cell indicates a selected example. Objectives are not systematically paired across bodies, so this coverage does not isolate morphology effects. Domain composition, overlapping requirements, and task checkers jointly define the scope of the results in Section 5.

5 Benchmarking Multimodal Agents on RobotWorld

We evaluate whether multimodal agents can complete physical tasks through the interfaces introduced in Section 3. We first compare success across the full benchmark and its five task domains, then examine the time and interactions used by each model.

5.1 Experimental Setup

Models and tasks. We evaluate GPT-6 Astra, Claude Opus 5.5, Kimi K3, DeepSeek V4.1 Flash and Gemini 3.8 Flash on all 84 tasks. The experiments were collected from 30 September to 6 October 2026, with one episode per model–task pair, giving 420 scored episodes. We use the same task initialisation protocols, control-step horizons and scoring rules across models.

Interaction settings. Each agent receives the task goal, permitted observations and tool documentation. It issues direct robot-tool calls and replans from the returned feedback; environment-side code control is disabled. Auxiliary computation and image inspection follow the permissions of each environment. The observation history contains the current observation and up to four earlier observations sampled every two observation rounds. Agents cannot access hidden checker state or reference solutions. Appendix B describes the available tools, and Appendix C gives representative system and developer instructions.

5.2 Evaluation Protocol and Metrics

Episode budgets. Each episode starts from the prescribed task state and ends when the task terminates or its budget is exhausted. Each task has a predefined control-step horizon. A separate non-action budget limits auxiliary calls, zero-step robot requests and otherwise action-free turns. On 83 tasks, the limits are 15 consecutive events and 30, 60 or 120 total events, depending on the task; volleyball 1v1 uses 20 and 150.¹ Executed control steps reset the consecutive counter, while API retries do not consume this budget. The numerical limits and accounting rules are detailed in Appendices B and D.

Task success. Completion is determined by the executable checks described in Section 4.3. We use the outcome at the applicable budget boundary, including retrospective adjudication when a recorded episode continued past that boundary. An unfinished task at the boundary counts as a failure. For a task set $\mathcal{G}$, the task-equal success rate of model m is

$$\text{SR}_{\mathcal{G}}(m) = \frac{1}{|\mathcal{G}|} \sum_{i \in \mathcal{G}} S_i(\xi_{i,m}), \quad (3)$$

where $\xi_{i,m}$ is the evaluated episode and $S_i \in \{0, 1\}$ is its task checker. Native rewards and partial scores are not averaged across tasks. Since each pair has one evaluated episode, the results describe this evaluation rather than variability over repeated trials.

Resource use. We measure elapsed wall-clock time, robot requests and all recorded tool calls. The last quantity includes robot requests, shell calls and explicit image-view calls; images already returned in robot feedback are not counted again. We report medians and interquartile ranges across tasks, including unsuccessful episodes. These measures describe the full recorded interaction, whereas task success is assessed at the evaluation boundary. Appendix D provides the numerical distributions and an additional breakdown of control steps and auxiliary calls.

¹The task-specific total-budget tiers were calibrated from Astra interaction records and applied across models.

5.3 Main Results

Task completion remains limited across all five models. As shown in Figure 1, Astra completes 16 of 84 tasks (19.0%), followed by Opus 5.5 with 13 (15.5%), Kimi K3 with 2 (2.4%), and DeepSeek V4.1 Flash and Gemini 3.8 Flash with 1 each (1.2%). Astra leads Opus 5.5 by three tasks, or 3.6 percentage points, but even the highest-scoring model leaves 68 tasks unfinished. The low completion rates indicate that access to observations, robot tools and auxiliary computation does not yet translate into reliable execution across the benchmark’s diverse embodiments and objectives.

Different models solve partly different tasks. Astra and Opus 5.5 share eight successes; Astra solves eight additional tasks and Opus 5.5 solves five. Their combined coverage is 21/84 tasks (25.0%). As shown in Figure 8, the successes of Kimi, DeepSeek and Gemini fall within this set, leaving 63 tasks unsolved by any evaluated model. Table 3 reports the five-domain results for every model. This union describes coverage by the evaluated models, rather than the performance of an agent that can select the best model in advance.

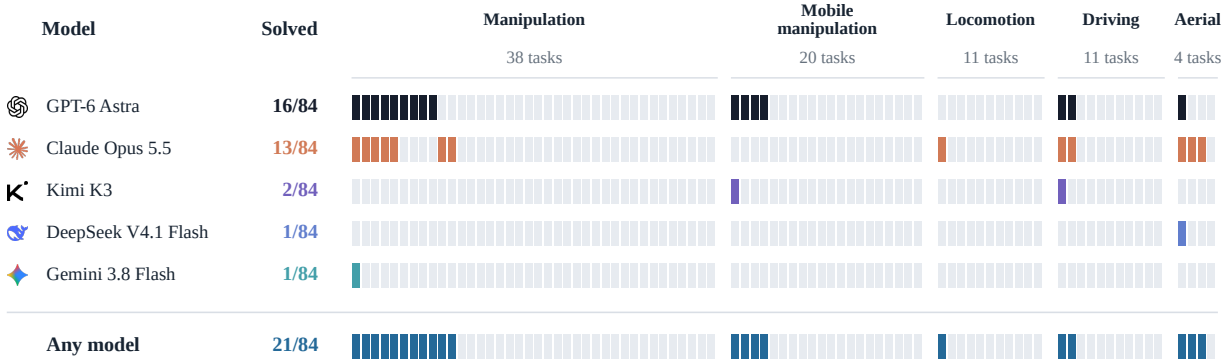


Figure 8 Success coverage across the 84 tasks. Each aligned column represents one task, grouped by its primary domain. Coloured cells indicate success and grey cells indicate failure. Tasks are ordered identically for all models, with successful tasks grouped within each domain. The bottom row marks tasks solved by at least one model: 21 tasks are covered, leaving 63 unsolved. Each model has one evaluated episode per task; counts use the outcomes at the applicable evaluation boundary.

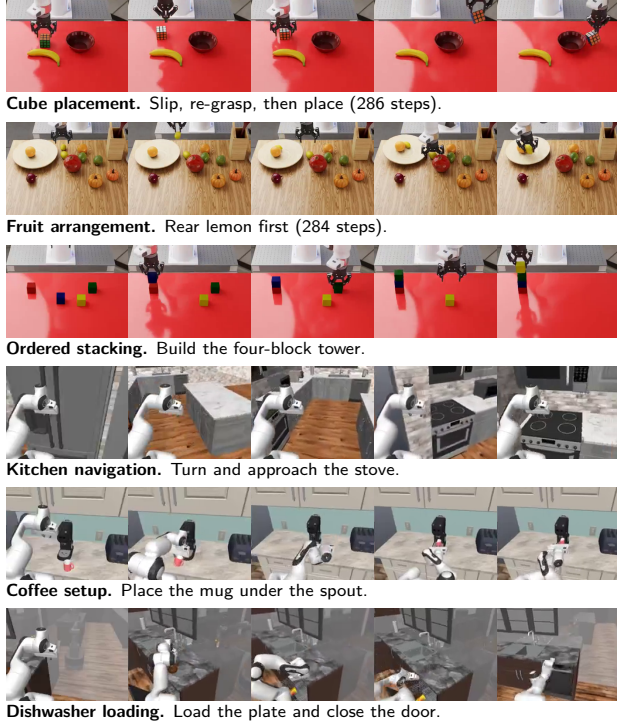
Table 3 Main results across five task domains. Entries give successful tasks / evaluated tasks; overall success rates are in parentheses. One episode is retained per model–task pair. Figure 1 pools the first two domains for compact presentation.

Model	Manip.	Mobile manip.	Locomotion	Driving	Aerial	Overall
Astra	9/38	4/20	0/11	2/11	1/4	16/84 (19.0%)
Opus 5.5	7/38	0/20	1/11	2/11	3/4	13/84 (15.5%)
Kimi K3	0/38	1/20	0/11	1/11	0/4	2/84 (2.4%)
DeepSeek V4.1 Flash	0/38	0/20	0/11	0/11	1/4	1/84 (1.2%)
Gemini 3.8 Flash	1/38	0/20	0/11	0/11	0/4	1/84 (1.2%)

The successful episodes cover a range of robot tasks. Astra completes tasks such as charger insertion, drawer closure and ordered block stacking, while Opus 5.5 succeeds at quadruped push recovery, payload hovering and drone juggling. Kimi succeeds at drawer closure and courtyard driving; Gemini succeeds at conveyor matching; DeepSeek succeeds at volleyball 1v1. Representative Astra and

Opus 5.5 episodes are shown in Figure 9. Appendix D lists all 420 outcomes, and Appendix E follows selected successes through observations, decisions and execution feedback.

Astra



Opus 5.5

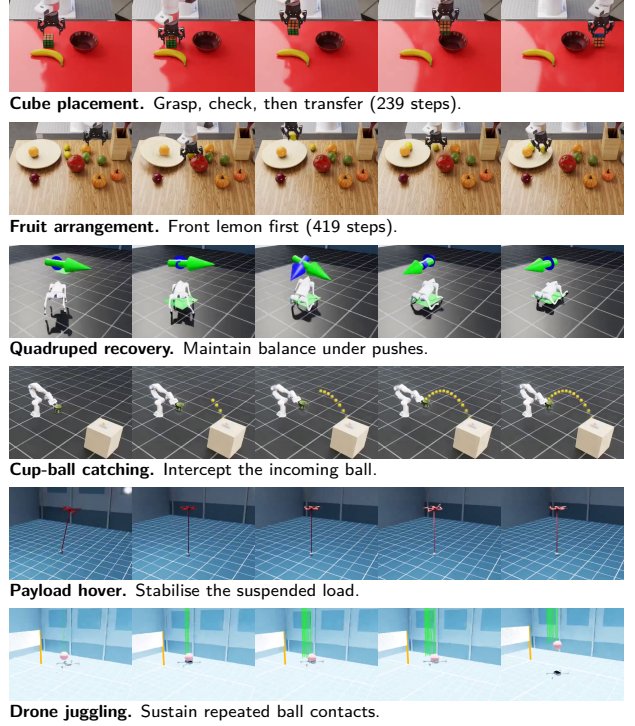


Figure 9 Representative successful trajectories of Astra (left) and Opus 5.5 (right). Each strip contains five chronological video frames. The first two rows compare shared successes; the remaining rows show tasks solved only by the displayed model in the illustrated trajectory comparison. Sampling intervals vary, and final frames may precede recorded success.

5.4 Performance Across Task Domains

Separating mobile manipulation reveals a larger gap between the leading models. As shown in Table 3, Astra solves 9/38 manipulation tasks (23.7%) and Opus 5.5 solves 7/38 (18.4%). In mobile manipulation, Astra completes 4/20 tasks (20.0%), while Opus 5.5 completes none. Kimi K3 completes the drawer-closing task, giving 1/20 (5.0%); DeepSeek and Gemini complete none. The combined panel in Figure 1 therefore pools two distinct result profiles: 13/58 for Astra and 7/58 for Opus 5.5.

Within fixed-base manipulation, Astra and Opus 5.5 share five successes. Astra solves four additional tasks, and Opus 5.5 solves two, giving a union of 11/38. Astra’s four mobile-manipulation successes bring the pair’s coverage in the two manipulation domains to 15/58. Gemini’s conveyor-matching success and Kimi’s drawer-closing success do not expand that union. The aligned task columns in Figure 8 expose these shared and model-specific successes without treating partially completed tasks as solved.

Dynamic-control successes favour Opus 5.5 on the evaluated tasks. Opus 5.5 completes 3/4 aerial tasks (75.0%), compared with 1/4 each for Astra and DeepSeek. It also records the only locomotion-domain success, quadruped push recovery, for 1/11 (9.1%). Its lower aggregate score therefore coexists with successes in domains where Astra is less successful. The aerial set is small:

one task changes the rate by 25 percentage points. These are task-level observations from a single retained episode per pair, rather than estimates of repeated-run reliability.

Driving and locomotion remain broadly unsolved. Astra and Opus 5.5 each complete courtyard and hairpin driving, yielding 2/11 (18.2%); Kimi completes courtyard driving, yielding 1/11 (9.1%). No evaluated model completes the remaining nine driving tasks or ten of the eleven locomotion tasks. Across all five models, the domain unions are 11/38 manipulation, 4/20 mobile manipulation, 1/11 locomotion, 2/11 driving and 3/4 aerial. The 63 tasks outside these unions show that the benchmark’s difficulty extends beyond a single embodiment or interface. Section 6 examines the execution mechanisms behind selected successes and failures.

5.5 Recorded Resource Consumption

Longer episodes do not consistently correspond to more successful tasks. As shown in Figure 10, median elapsed times are 30.9 minutes for Astra, 54.0 for Opus 5.5, 67.6 for Kimi, 16.0 for DeepSeek and 12.8 for Gemini. Astra achieves the most successes with a shorter median episode than either Opus 5.5 or Kimi. Kimi has the longest median duration but completes only two tasks. These aggregate differences show that time spent interacting with the environment alone does not explain the success ranking.

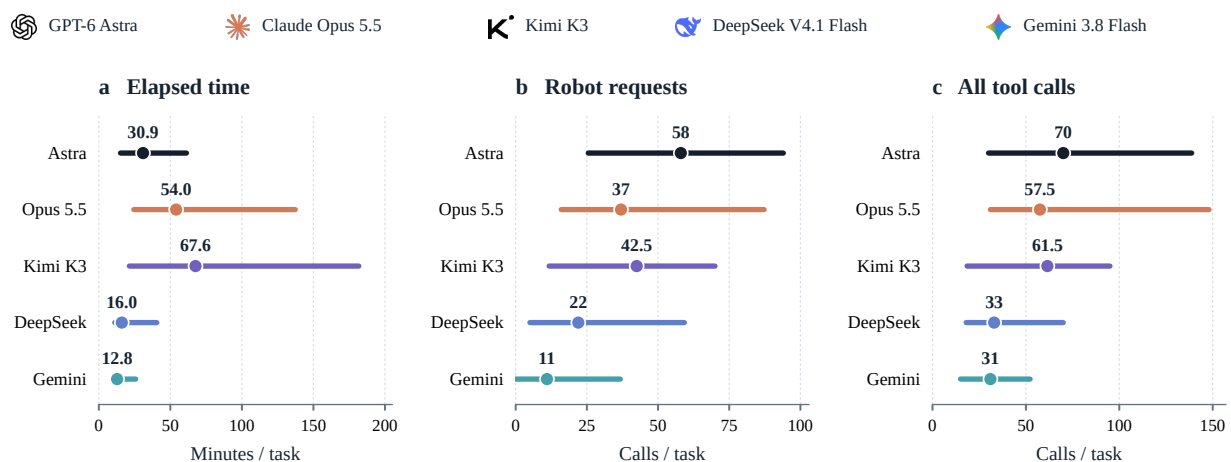


Figure 10 Recorded resource consumption. Points and labels show medians; horizontal segments span the 25th–75th percentiles across 84 retained runs per model, including failed runs. Models use the same colours and row order in every panel. Elapsed time includes setup, waiting and execution. Total tool calls include robot requests, shell calls and explicit image-view calls. Full logs may extend beyond retrospective adjudication cutoffs; the intervals describe variation across tasks, rather than uncertainty over repeated trials.

Interaction frequency and elapsed time give different orderings. Median robot-request counts are 58, 37, 42.5, 22 and 11 for Astra, Opus 5.5, Kimi, DeepSeek and Gemini, respectively; median totals including auxiliary tools are 70, 57.5, 61.5, 33 and 31. Astra therefore makes more robot requests per median episode than Opus 5.5 or Kimi despite its shorter median elapsed time. Across tasks, durations also vary widely: the interquartile range is 15.2–61.2 minutes for Astra, 24.5–137.3 for Opus 5.5 and 21.3–181.7 for Kimi. Call counts alone do not capture these timing differences.

Elapsed time includes setup, waiting and execution, and full logs may extend beyond a retrospectively adjudicated cutoff. In addition, short episodes can reflect early failure or stopping. These

measurements therefore describe resource consumption, rather than time to success or a ranking of efficiency. The matched-trajectory analyses in Section 6 and Appendix G examine resource use for comparable task outcomes.

5.6 Success, Token Consumption, and Estimated Cost

We compare the overall success rate with three aggregate resource measures in Figure 11: estimated API cost at list prices, total tokens, and cumulative elapsed time. All three measures use the project website’s aggregate accounting: the mean over scored tasks with a valid recorded value, multiplied by 84. Missing or zero-flagged token usage is excluded from the mean; Kimi’s token and cost estimates therefore extrapolate from 57 valid records, while the other models have 84. Tokens include cached input and output, and costs are list-price estimates rather than invoices. All 84 elapsed durations are available for each model; these website-reported durations include simulation, API latency and waiting, rather than the duration of a parallel evaluation campaign. Appendix D gives the aggregate values and accounting scope.

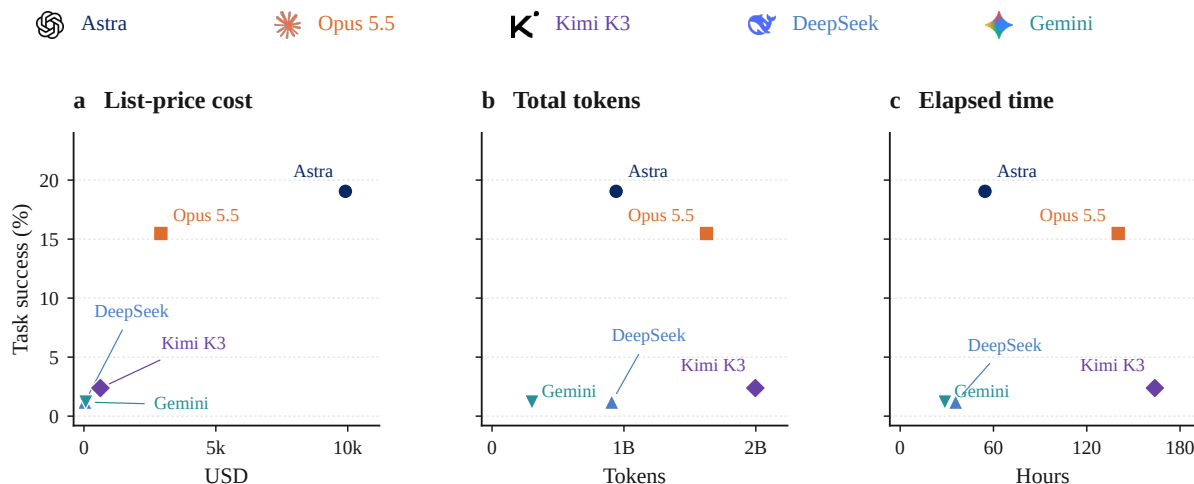


Figure 11 Success versus aggregate resource consumption. Each point represents one model evaluated on 84 tasks; all panels use the same success axis. (a) Estimated list-price cost. (b) Total tokens, including cached input and output. (c) Cumulative elapsed time, including unsuccessful runs. All values follow the project website: valid per-task mean multiplied by 84. Token and cost estimates use 57 valid records for Kimi and 84 for each other model; all elapsed-time totals use 84 records. Missing usage is estimated by this scaling, rather than counted as zero. Resource accounting covers full recorded runs and may extend beyond retrospective scoring cutoffs; these plots do not measure cost or time to first success.

The strongest result has the highest reported cost, but not the largest token volume. Astra reaches 19.0% success at a reported \$9,913 and 941M tokens. Opus 5.5 reaches 15.5% at \$2,916 and 1.63B tokens. Thus Astra’s estimated expenditure is about 3.4 times Opus 5.5’s, while its token volume is lower. Different model prices and cache accounting prevent raw token totals from being interpreted as a common monetary budget. Kimi K3 consumes the most reported tokens (2.00B), with \$624 estimated cost, but completes only two tasks. DeepSeek records one success at 907M tokens and \$34; Gemini records one at 303M tokens and approximately \$67.

More aggregate computation or elapsed time does not ensure broader task coverage. Cumulative elapsed times are 54.6 hours for Astra, 140.3 for Opus 5.5, 163.8 for Kimi, 35.7 for DeepSeek and 28.7 for Gemini. Astra completes the most tasks while using less elapsed time than

Opus 5.5 or Kimi; Kimi consumes the most time and tokens without comparable success. These comparisons describe the evaluated model–harness combinations. They do not isolate inference speed, hardware, pricing effects or a causal benefit of extra computation. In particular, an early failed episode can consume few resources. The matched successful-task comparisons in Section 6 therefore complement these aggregate plots by conditioning on shared completed tasks.

6 Analysis

The following case studies and cross-task profiles use the same retained five-model evaluation reported in Section 5. We relate decisions to recorded execution and evaluator outcomes. Success uses the applicable evaluation boundary; resource distributions describe full retained logs, while explicitly identified milestone comparisons use matched progress within an episode. Paired Astra–Opus 5.5 cases examine different routes to comparable progress, and Kimi and DeepSeek cases expose failures in state tracking and continuation. These examples establish mechanisms in individual episodes, rather than their frequency across the benchmark.

6.1 Feedback-Driven Adaptation and Recovery

Figure 12 illustrates action refinement, reconfiguration after rejection, and sustained recovery. In K3’s drawer-closing episode, reduced end-effector displacement prompted shorter pushing segments, followed by withdrawal to check closure. Native success occurred during withdrawal: the useful pattern was feedback-dependent refinement and verification.

Recovery can require changing the configuration from which an action is attempted. In charger insertion, Astra tried rotating the idle arm’s wrist camera downward to inspect alignment. The planner rejected the command at step 196. Astra then lowered and brought the idle arm nearer before retrying the same requested pitch; this rotation executed successfully, as shown in Figure 12c. Further insertion adjustments, release and withdrawal produced native success at step 323. This sequence illustrates feedback-dependent reconfiguration, although it does not isolate that change as the cause of final success.

Dynamic recovery additionally requires sustaining the recovered state. Opus 5.5 combined offline linear-quadratic regulator calculations with repeated state feedback in drone payload hover. Payload error decreased, and the final interval met the joint position, velocity, swing, and attitude conditions, as shown in Figure 12b. Here, auxiliary computation supported successful execution through commands checked against subsequent observations.

Takeaway

Feedback matters when it changes execution. Effective recovery adapts the command, revises a failing strategy, and verifies that the recovered state persists.

6.2 Failure Modes

In K3’s pouring episode, the initial approach knocked the bottle onto its side. The agent then reported that its first closure missed the fallen bottle. Repeated left-arm attempts did not establish a grasp; at step 240 it switched to the right arm, whose first repositioning ended at step 272. The remaining budget was consumed by further approach motions, ending without success at step 400, as shown in Figure 13a. Reaching successive arm poses did not restore the object state needed for pouring.

Preparation can also consume the entire horizon. All 69 positive-step calls in K3’s vegetable-chopping episode concerned surveying, navigation, or clearance around furniture. The final command

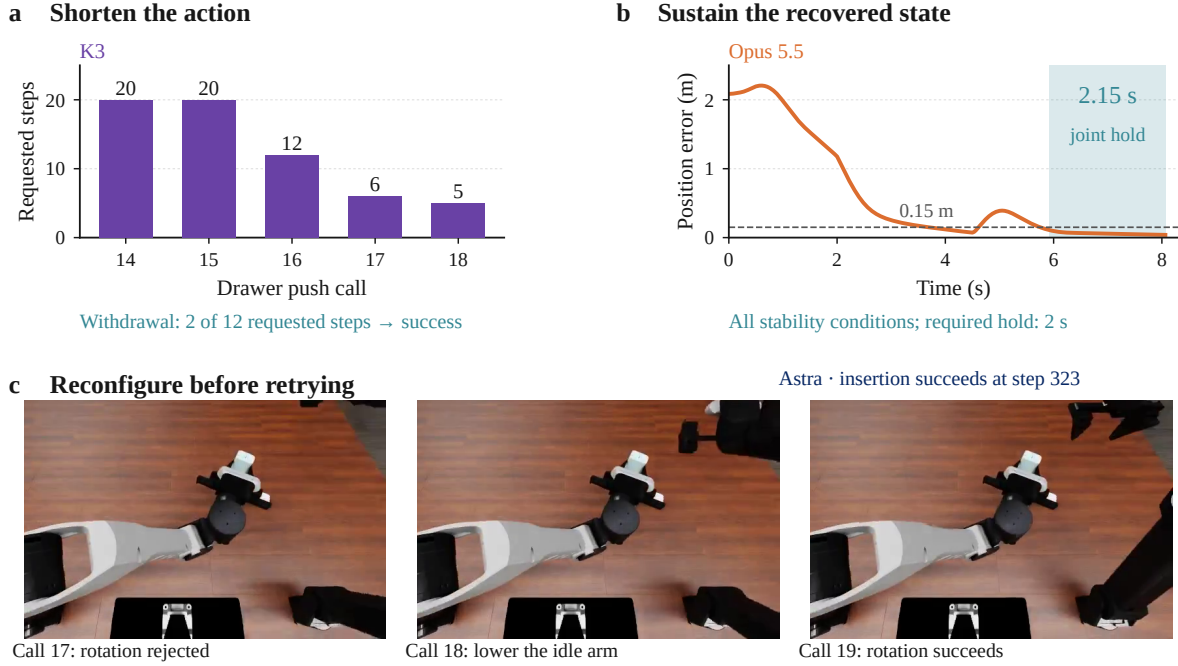


Figure 12 Feedback-dependent correction and its limits. (a) Kimi K3 shortens successive drawer pushes; withdrawal executes two of twelve requested steps before success. (b) Opus 5.5 sustains payload hover: shading marks all stability conditions jointly satisfied for 2.15 seconds, exceeding the two-second requirement. (c) Astra changes the idle arm’s position after a downward camera rotation is rejected at step 196: it lowers the arm by step 207 and successfully retries the rotation by step 230. The episode subsequently completes charger insertion at step 323. Frames show the corresponding states from the retained recording; the rejected call itself executes no steps.

still attempted kitchen entry; no cutting command appeared before the 2,000-step limit; see Figure 13c. The observed bottleneck was reaching the workspace, so this failure does not diagnose the unattempted cutting skill.

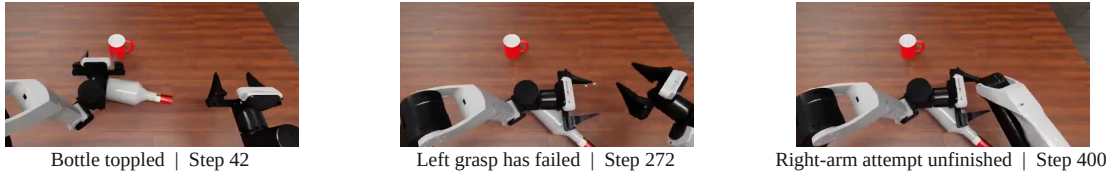
Misjudging completion can similarly prevent further correction. In block sweeping, K3 described the task as complete after returning the empty left arm home at step 861. Its final 69 calls changed only that arm’s vertical target, advancing another 139 control steps without resuming sweeping. Native success remained false at the 1,000-step horizon, as shown in Figure 13c. This episode includes earlier rejected commands and revised targets, but the terminal failure is continued execution without corrective task action.

Recovery must also arrive in time. Astra’s humanoid table-tennis episode ended at 3.58s when its base fell below 0.50m despite balance corrections. In inverted-pendulum tracking, Astra completed the horizon with only 2.45cm final tip error, but violated the tracking tolerance when checking began at 0.5s. Its final stable interval was 0.565s rather than the required second; see Figure 13b. A favourable final frame can therefore conceal temporal failure.

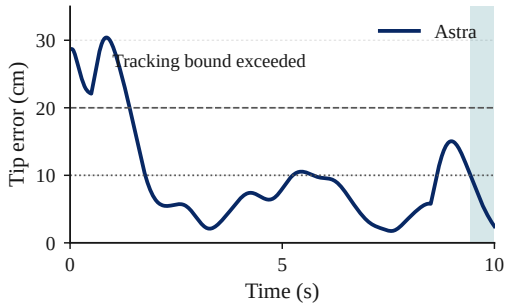
Takeaway

Motion is not task progress. Failures include losing the required object state, remaining in preparation, mistaking an unfinished state for completion, and recovering too late.

a Object loss precedes a late strategy change

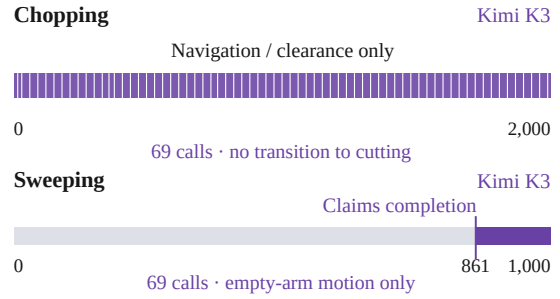


b Recovery comes too late



Final hold: 0.565 s; required: 1 s

c Where task progress stops



Executed control steps

Figure 13 Execution without task completion. (a) Kimi K3 topples the bottle during approach, fails to secure it with the left arm, and switches to a right-arm attempt late in the 400-step episode. Images are sampled from the current run at the labelled control steps. (b) Astra crosses the tracking bound (20 cm) before recovering below the terminal bound (10 cm); shading marks all terminal conditions, including speed and tilt. The final joint hold lasts only 0.565 seconds. (c) Chopping never advances to the next subgoal: 69 positive-step calls cover the entire 2,000-step episode. Sweeping ends with 69 calls that vary only the empty left arm’s height over the final 139 steps, while the model asserts completion and the native task remains unsuccessful. Each bar spans its own episode horizon. White dividers mark chopping calls; the purple sweeping segment marks the final 139 steps. The chopping count covers the whole episode; the sweeping count covers only this final segment.

6.3 Control Strategies Across Tasks

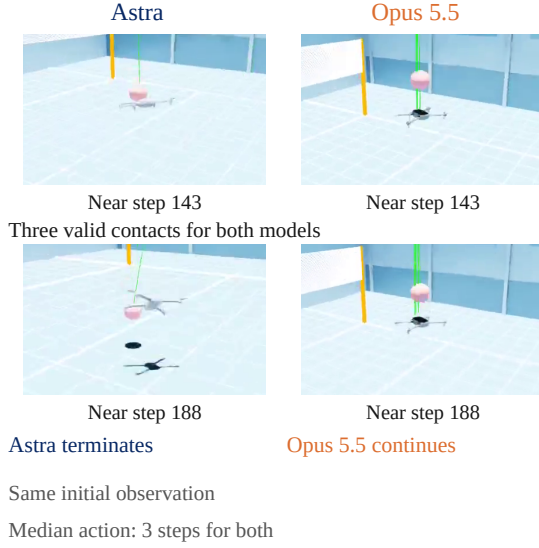
Figure 14 compares sustained contact control in drone juggling; Figure 15 shows the different orderings of perception, computation and action in stacking.

Both models performed offline dynamics calculations for drone juggling. Astra used position and attitude feedback with rotor thrust allocation. Opus 5.5’s short rotor-action segments levelled the vehicle, accelerated before impact, reduced thrust after contact, and repositioned beneath the ball. Opus 5.5 sustained 15 valid hits, including 14 height-qualified hits, over 800 steps; Astra recorded three valid hits, one height-qualified, before ball-too-low termination at step 188. Both had a median of three executed steps per call. The distinction is sustained contact coordination, not simply building a model or using shorter segments.

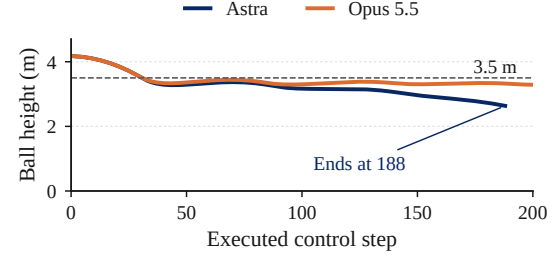
Stacking contrasts direct visual estimation with programmatically assisted localisation. Astra approached the blue block from successive camera observations, closed the gripper at step 157, and lifted at 179 before external image-processing or camera-fitting computation. It introduced a camera-projection fit only at step 204, after carrying blue towards red. Opus 5.5 wrote colour-segmentation code before moving, then sampled robot motions to fit camera geometry and compute targets.

Astra reused the grasp geometry and stack centre for later blocks, checking the resulting state

a Matched trajectory frames



b Ball height before termination



c Cumulative contact outcomes

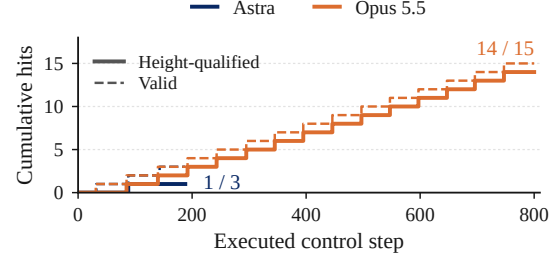


Figure 14 Sustaining contact under the same initial observation. (a) Matched frames from Astra and Opus 5.5 near steps 143 and 188. Both have made three valid contacts near step 143; Astra terminates at 188 while Opus 5.5 continues. (b) Ball height over the first 200 steps; the dashed line marks the 3.5 m qualification threshold. (c) Valid and height-qualified contact counts, ending at each trajectory’s termination. The first contact is not height-qualified. Astra ends with 3 valid / 1 qualified contacts; Opus 5.5 reaches 15 / 14 at step 800. Both use a median of three executed steps per action. Frames are nearest samples from 12-fps recordings.

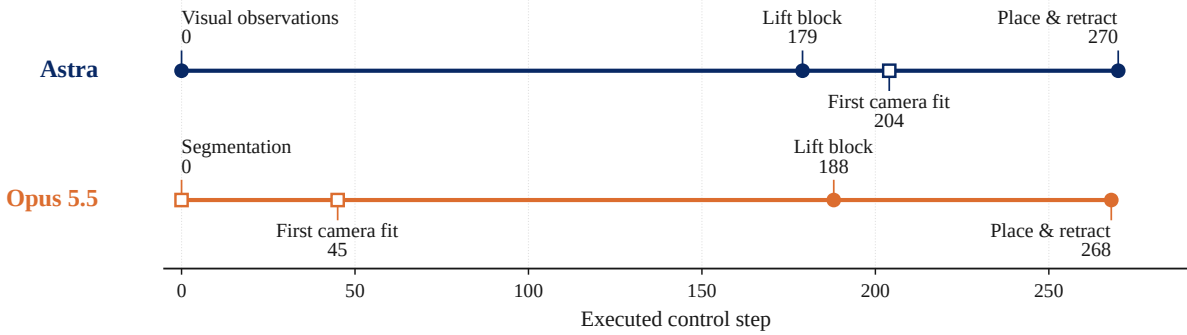


Figure 15 Different routes from observation to grasping. Milestones from the two stacking episodes on a shared control-step axis. Filled circles mark observations or physical milestones; open squares mark explicit computation, during which physics is paused. Astra closes the gripper at step 157 and lifts at 179, before its first camera fit at 204. Opus 5.5 writes colour-segmentation code before motion, samples the arm and fits camera geometry at step 45, then lifts at 188. Both reach first placement and retraction at similar physical steps (270 versus 268), but their information-gathering sequences differ. Astra completes the stack at step 635; Opus 5.5 reaches the applicable auxiliary-call budget at step 313. These observations describe strategies in the illustrated episodes and do not identify the models’ training data.

through lifts and retractions. Opus 5.5 repeatedly refined segmentation and geometric estimates. Astra finished in 635 control steps; Opus 5.5 reached its auxiliary budget at step 313 without completion. Section 6.5 compares the interaction costs of their shared first-block milestone.

Takeaway

Useful estimates must become sustained control. Both models build numerical controllers; their trajectories differ in coordinating repeated contacts and in obtaining and reusing spatial estimates.

6.4 Completion Assessment and Decisions to Continue

Table 4 distinguishes these judgements using stated assessments and subsequent actions. The relevant distinction is whether the next command tests or repairs an unresolved task condition: execution can continue even after corrective search has stopped.

Table 4 Completion judgements and subsequent execution. Holding and return-to-home commands advance simulation even when they do not advance the unfinished task. In white-mug placement, the tool closes without a final observation or success score; the release request may itself trigger success and does not establish post-success over-action.

Case / model	Agent judgement	Subsequent behaviour	Recorded result
Block sweeping Kimi K3	Declares completion at step 861	69 empty-arm height commands; 139 more steps	Unfinished at the 1,000-step horizon
Stove navigation DeepSeek	No further motion needed at step 345	105 more steps with zero base-velocity commands	Unfinished at the 450-step horizon
Hanging mugs Opus 5.5	35 steps deemed insufficient to grasp and hang a mug	Returns the arm home over 35 steps	Unfinished at the 800-step horizon
White mug Opus 5.5	Requests gripper release at step 340	One of 15 requested steps executes	Native success at step 341

In DeepSeek’s stove-navigation episode, visibility, proximity, and apparent counter contact supported its claim of completion. A continuation prompt elicited repositioning, but the model then judged further motion unwarranted and issued holding commands. The final 105 steps advanced simulation using zero base-velocity commands, while native success remained false. Continuation restored tool use without restoring corrective search. Kimi’s sweeping episode exhibits the same distinction through nonzero motion: the empty arm changes height, but those commands do not revisit the unfinished sweeping objective, as shown in Figure 13c.

Opus 5.5 instead recognised failure during mug hanging: opening the gripper dropped the yellow mug. It first returned home, then made another approach with 83 steps remaining. At step 765, it judged the final 35 steps insufficient to grasp and hang a mug and spent them returning the arm home. This was a judgement of infeasibility, not a claim of success; the trace does not establish that recovery was impossible.

Successful termination leaves a different ambiguity. Opus 5.5 requested a 15-step release after observing step 340 in white-mug placement; success occurred at step 341, followed by closure without a final observation or score. The release may itself have triggered success, so the plan does not establish unnecessary post-success action. In this placement episode, native success ends execution, whereas sustained-stability tasks require continued control beyond an instantaneously favourable

state. A continuation rule must therefore distinguish the agent’s own assessment from the task’s actual termination and temporal requirements.

Takeaway

Continuation requires judging both completion and recoverability. An agent can stop correcting because it believes the goal is met, or because it believes further progress is infeasible.

6.5 Interaction Efficiency

Figure 16 separates robot, shell, and explicit image-view calls from executed control steps for matched scenes and seeds. Images embedded in robot feedback and accompanying prose add no separate call. These are execution counts, independent of API latency.

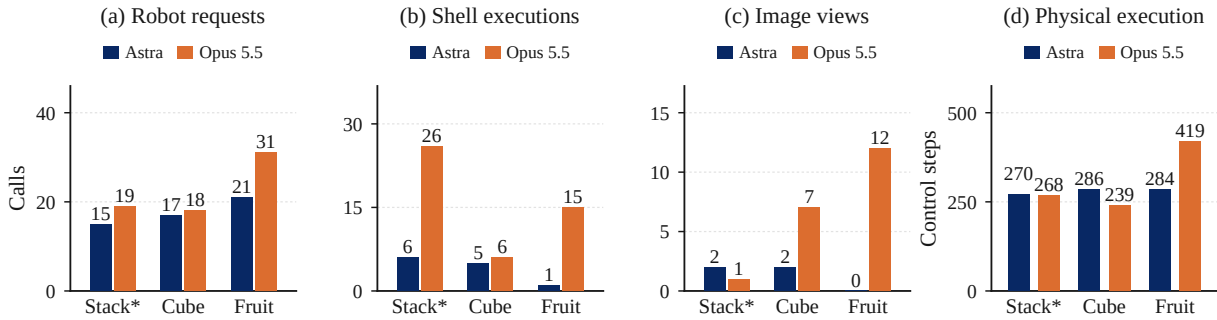


Figure 16 Tool calls and physical execution measure different costs. Stacking ends at the shared first blue-on-red placement and retraction, before Opus 5.5’s step-313 budget cutoff. Cube and fruit comparisons cover full successful episodes. Separate panels count robot requests, shell executions, explicit image-view calls and control steps. Total tool calls (Astra / Opus 5.5) are 23 / 46 for stacking, 24 / 31 for cube and 22 / 58 for fruit. Images embedded in robot feedback are not additional image-view calls.

For first-block placement and retraction, Astra used 23 tool calls versus Opus 5.5’s 46, despite nearly identical physical execution (270 versus 268 steps). Before lifting blue, Astra used ten robot calls and two shell calls; Opus 5.5 used 13 robot calls and 25 auxiliary calls. The localisation strategies in Section 6.3 thus reached comparable progress at different interaction costs, including analysis and software preparation.

Cube placement reverses the physical-cost ordering. Astra’s initial grasp slipped, requiring realignment and a deeper grasp. Opus 5.5 performed additional localisation and contact checks without the same drop-and-regrasp sequence. Both succeeded: Astra used fewer tool calls (24 versus 31), but more control steps (286 versus 239).

In fruit arrangement, both costs increased together: Astra used 22 tool calls and 284 steps; Opus 5.5 used 58 calls and 419 steps, including extra contact-relief motions, gripper checks, and placement adjustments. Since the models chose different lemons first, the comparison uses full successful episodes. Reporting both costs reveals how information gathering and physical recovery contribute to reaching a matched outcome.

Takeaway

Interaction cost and physical cost can diverge. Comparable progress may require twice as many tool calls; fewer tool calls need not mean fewer control steps.

6.6 Cross-Task Model Profiles

Across 84 matched task records, Figure 17 compares interaction distributions and Figure 19 groups outcomes by task requirements.

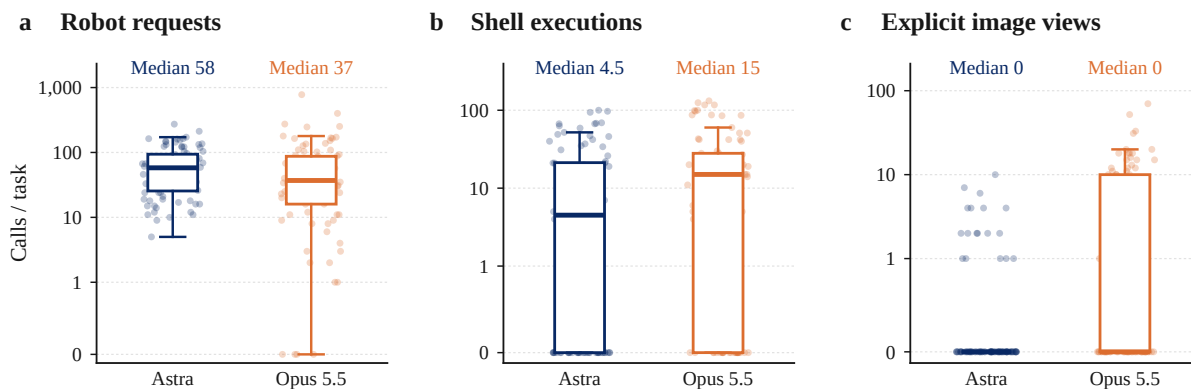


Figure 17 Interaction distributions across 84 tasks. Points represent the full retained episode for each task, including failures; boxes show medians and interquartile ranges, with whiskers extending to the most extreme points within 1.5 interquartile ranges. Robot requests include rejected and zero-step requests; shell counts exclude polling. Explicit image views exclude images embedded in feedback. Astra has zero explicit views on 66/84 tasks and Opus 5.5 on 43/84, so both medians are zero; their upper quartiles are 0 and 10 calls, respectively. Symmetric-log axes retain zeros. These distributions describe tool use, not efficiency or success-conditioned cost.

In the full retained logs, Opus 5.5 makes 2,095 shell calls and 568 explicit image-view calls, compared with Astra’s 1,435 and 56. Median shell counts are 15 versus 4.5 per task. Thus, external computation and explicit inspection are more common in Opus 5.5’s recorded interactions. Counts alone do not reveal how useful those calls were: Figure 15 supplies the missing ordering and purpose for one matched task.

Among eight tasks solved by both models, Astra uses fewer tool calls in seven and fewer control steps in six, as shown in Figure 18. Conveyor grasping reverses both orderings; cube placement reverses the step ordering. This extends Section 6.5 beyond individual examples. Across all tasks, call totals also reflect the duration of unsuccessful attempts and are not an efficiency ranking.

Five overlapping goal labels cover all tasks: spatial placement/navigation, constrained contact, continuous stabilisation, timed interaction, and multiple subgoals. Ordinary approach–grasp–lift alone does not qualify as multiple subgoals.

Astra succeeds on 13/45 spatial tasks versus Opus 5.5’s 6/45, and on 6/41 constrained-contact tasks versus 1/41. Opus 5.5 leads on continuous balance/tracking (4/20 versus 1/20) and timed interaction (4/7 versus 2/7), including quadruped push recovery and cup-based ball catching as well as drones. Overlapping groups describe related views of the same task set.

Takeaway

The model profiles differ across task demands. Astra uses fewer tool calls on seven of eight jointly solved tasks and succeeds more often on spatial/contact goals; Opus 5.5 succeeds more often on continuous and timed dynamic goals.

We hypothesise that these profiles reflect different ways of composing perception, computation, and control. Astra may translate visual observations into spatial estimates and actions more effectively,

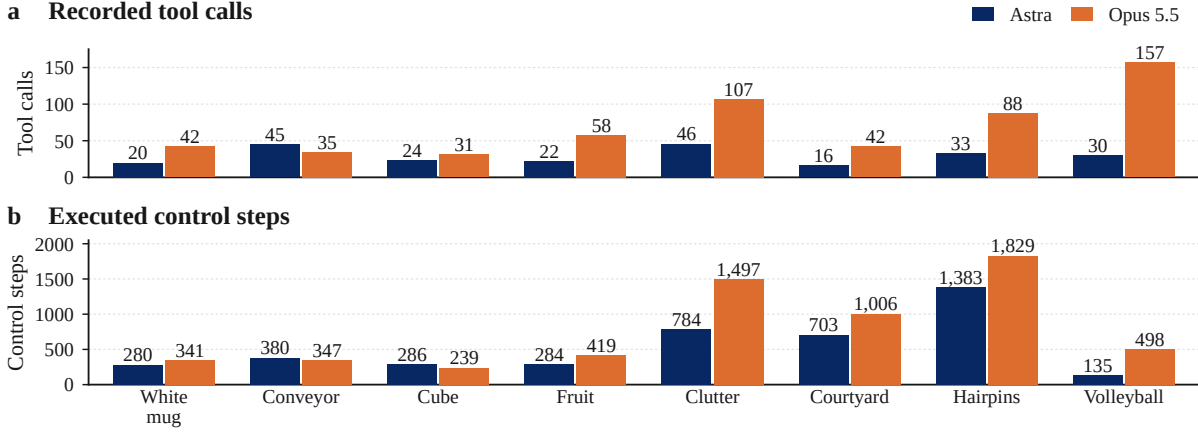


Figure 18 Resource use on the eight tasks solved by both models. Absolute tool-call and control-step counts cover each full retained successful episode. Tool calls include robot requests, shell executions and explicit image views; control steps use the final episode count. Astra uses fewer tool calls on seven tasks and fewer control steps on six. These paired costs condition on shared success; the all-task distributions in Figure 17 include unsuccessful runs.

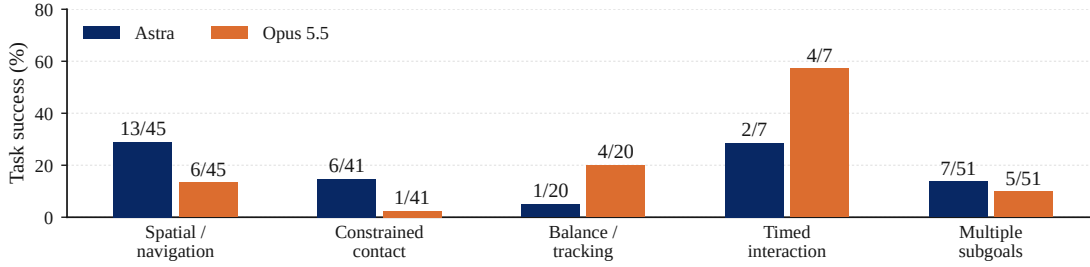


Figure 19 Exploratory task-demand profiles. Bars show success rates with exact counts. Labels overlap and both models are evaluated on the same tasks within each group. All 84 tasks are scored for both models. Group membership is assigned from task requirements, not inferred from the outcome; these descriptive differences do not isolate component abilities.

reducing external inspection during placement and alignment. Opus 5.5 more often expresses geometric or physical relationships through programs, which may aid prediction and timing in some dynamic tasks. Both routes occur in both models: Astra also builds numerical controllers, and successful Opus 5.5 control need not use many shell calls. Robot use may therefore draw on the transfer and composition of visual-spatial reasoning, program construction, and closed-loop agent execution. This interpretation remains an exploratory hypothesis rather than an isolated component measurement. The comparisons do not identify training-data differences, and neither tool volume nor a single successful strategy establishes a general capability advantage.

Taken together, the cases identify three concrete requirements for reliable robot use: preserve the object and subgoal state across motions, turn failure feedback into a changed task-directed action, and maintain the required conditions for the full evaluation interval. Figures 12 and 13 show why local pose accuracy alone is insufficient; Figures 15 and 18 show why useful computation must also be assessed against physical progress and matched outcomes.

7 Discussion and Limitations

Our results suggest that progress in robot use depends on how agents combine capabilities throughout an episode. Visual estimation, program construction, and dynamics reasoning can each support useful actions, yet reliable completion requires the agent to track their consequences, revise a failing strategy, and assess progress towards the goal. This motivates training and agent designs that strengthen the connection between perception, computation, and feedback-driven execution. The contrasting model profiles offer hypotheses about where these connections differ; they do not isolate the contribution of vision, coding, or control. Targeted comparisons that vary access to visual observations, auxiliary computation, and controller support could test these hypotheses and identify which interventions improve task completion.

The transition from simulation to physical deployment remains an open test. Our results measure performance under the observations and control interfaces available in each task. These interfaces provide different degrees of assistance, and pausing simulation during model inference removes an important real-world timing constraint. Physical robots must act despite inference delays, sensor noise, and discrepancies in contact and dynamics. Evaluating the same agent strategies in real-time loops and on hardware would establish which capabilities survive these demands and where faster feedback or additional control support is needed.

The suite spans diverse tasks and embodiments, but its coverage is finite and uneven, with related tasks inherited from shared source projects. Results therefore characterise performance on this suite rather than a representative distribution of all robot use. Public task descriptions, code, or demonstrations may also have appeared in model training data. Extending evaluation to newly constructed tasks, varied initial conditions, and additional embodiments would test whether the observed strategies transfer beyond familiar settings and help distinguish reusable physical competence from task-specific success.

8 Conclusion

RobotWorld provides a challenging, systematic proving ground for agents moving from digital tasks towards robot use. Across diverse tasks and embodiments in simulation, our analysis reveals both the scope of emerging capabilities and the difficulty of composing them into reliable physical action. Agents can construct perception and control workflows, estimate spatial relationships, reason about dynamics, and revise actions from feedback. Yet successful execution also requires preserving task-relevant states, correcting ineffective actions, meeting temporal constraints, and recognising when a goal has actually been achieved. Differences across models further show that strengths in one class of physical demands do not imply uniformly strong robot use. These findings motivate progress in how perception, computation, and action work together throughout an episode. By connecting measurable outcomes with observable execution patterns, RobotWorld gives the community a basis for identifying training and agent-design priorities and testing whether proposed improvements lead to more reliable robot behaviour. We hope this shared proving ground helps turn the promise of physical-world agents into cumulative, measurable progress.

References

- Anthropic. How Claude performs on robotics tasks, 2026a. <https://www.anthropic.com/research/claude-plays-robotics>.
- Anthropic. The future of AI at work: Introducing Cowork, 2026b. <https://www.anthropic.com/webinars/future-of-ai-at-work-introducing-cowork>.
- Mickyas Tamiru Asfaw. Wheeled Quadruped Robot: Deep RL in Isaac Lab, 2026. https://github.com/MickyasTA/wheeled_quadruped_robot.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. <https://arxiv.org/abs/2410.24164>.
- Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054*, 2025. <https://arxiv.org/abs/2504.16054>.
- BrandoUlissi. Isaac Lab – Go2 Locomotion. Software repository, n.d. <https://github.com/BrandoUlissi/isaacclab-go2-locomotion>. Accessed 6 October 2026.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. <https://arxiv.org/abs/2307.15818>.
- Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Weijie Wan, Baijun Chen, Haoran Lu, Haowen Yan, Honghao Su, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026a. <https://arxiv.org/abs/2607.04434>.
- Yanzhe Chen, Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Qiming Huang, and Mike Zheng Shou. Show-Harness: Just a VLM agent can play robots. *arXiv preprint arXiv:2609.10522*, 2026b. <https://arxiv.org/abs/2609.10522>.
- Yuxuan Chen, Wanruo Zhang, and Xiao Li. Reflex: Enabling Fast and Predictive Vision-Language-Action Models for Reaction-Critical Manipulation. *arXiv preprint arXiv:2608.14379*, 2026c. <https://arxiv.org/abs/2608.14379>.
- Zhili Cheng, Yuge Tu, Ran Li, Shiqi Dai, Jinyi Hu, Shengding Hu, Jiahao Li, Yang Shi, Tianyu Yu, Weize Chen, Lei Shi, and Maosong Sun. EmbodiedEval: Evaluate multimodal LLMs as embodied agents. *arXiv preprint arXiv:2501.11858*, 2025. <https://arxiv.org/abs/2501.11858>.
- Jay Chooi. GPT-6 Astra robot-control experiment. https://x.com/chooi_jeq/status/2096064315115839904, 2026. X post, September 5, 2026. Accessed October 4, 2026. Descriptive title.
- Jiafei Duan. Real-time rollout of GPT-6 Astra controlling MolmoAct2 YAMs. <https://x.com/DJiafei/status/2104068182843761008>, 2026. X post, September 27, 2026. Accessed October 4, 2026. Descriptive title.
- Ziqi Fan. robot_lab: RL Extension Library for Robots, Based on IsaacLab., 2024. https://github.com/fan-ziqi/robot_lab.
- Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Fei-Fei Li, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Jim Fan. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026. <https://arxiv.org/abs/2603.22435>.
- Max Fu. Multimodal robot control, harnesses, and tool calls. https://x.com/letian_fu/status/

- 2096673034325381268, 2026. X post, September 2026. Accessed October 4, 2026. Descriptive title.
- Galbot Team, Xuchuan Chen, Xiaoqian Cheng, Yu Deng, et al. Systematically exploring the capabilities of GPT-6 Astra as embodied policies. *arXiv preprint arXiv:2609.38537*, 2026. <https://arxiv.org/abs/2609.38537>.
- Tyler Han, Preet Shah, Sidharth Rajagopal, Yanda Bao, Sanghun Jung, Sidharth Talia, Gabriel Guo, Bryan Xu, Bhaumik Mehta, Emma Romig, Rosario Scalise, and Byron Boots. Demonstrating WheeledLab: Modern Sim2Real for Low-cost, Open-source Wheeled Robotics, 2025. <https://arxiv.org/abs/2502.07380>.
- Muqun Hu, Wenxi Chen, Wenjing Li, Falak Mandali, Zijian He, Renhong Zhang, Praveen Krisna, Katherine Christian, Leo Benaharon, Dizhi Ma, Karthik Ramani, and Yan Gu. PACE: Physics Augmentation for Coordinated End-to-end Reinforcement Learning toward Versatile Humanoid Table Tennis, 2026. <https://arxiv.org/abs/2509.21690>.
- Anlun Huang, Zhenyu Wu, Soofiyan Atar, Yuheng Zhi, and Michael Yip. SteadyTray: Learning Object Balancing Tasks in Humanoid Tray Transport via Residual Reinforcement Learning, 2026. <https://arxiv.org/abs/2603.10306>.
- Kiki Huang. GPT-6 Astra grasping an apple tip (open source). <https://www.xiaohongshu.com/explore/6aa27b3a000000002b025d03>, 2026. Xiaohongshu post (Chinese); edited September 21, 2026. Accessed October 4, 2026. Descriptive title.
- Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 540–562, 2023. <https://proceedings.mlr.press/v229/huang23b.html>.
- Phillip Isola. High-frequency controllers as tools for robot-use agents. https://x.com/phillip_isola/status/2097152728648569008, 2026. X reply, September 8, 2026. Accessed October 4, 2026. Descriptive title.
- Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J. Davison. RL Bench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 2020. <https://arxiv.org/abs/1909.12271>.
- jaykorea. Isaac LAB for Flamingo. Software repository, n.d. <https://github.com/jaykorea/Isaac-RL-Two-wheel-Legged-Bot>. Accessed 6 October 2026.
- Shilong Ji, Yinuo Chen, Chuqi Wang, Jiayu Chen, Ruize Zhang, Feng Gao, Wenhao Tang, Shu’ang Yu, Sirui Xiang, Xinlei Chen, Chao Yu, and Yu Wang. JuggleRL: Mastering Ball Juggling with a Quadrotor via Deep Reinforcement Learning, 2025. <https://arxiv.org/abs/2509.24892>.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. <https://arxiv.org/abs/2406.09246>.
- Jipeng Kong, Xinzhe Liu, Yuhang Lin, Jinrui Han, Sören Schwertfeger, Chenjia Bai, and Xuelong Li. Learning Soccer Skills for Humanoid Robots: A Progressive Perception-Action Framework, 2026. <https://arxiv.org/abs/2602.05310>.
- Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, et al. BEHAVIOR-1K: A human-centered, embodied AI benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024a. <https://arxiv.org/abs/2403.09227>.
- Manling Li, Shiyu Zhao, Qineng Wang, Kangrui Wang, Yu Zhou, Sanjana Srivastava, Cem Gokmen, Tony Lee, Li Erran Li, Ruohan Zhang, et al. Embodied agent interface: Benchmarking LLMs for embodied decision making. In *Advances in Neural Information Processing Systems*, 2024b. <https://arxiv.org/abs/2410.07166>.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022. <https://arxiv.org/abs/2209.07753>.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.

- <https://arxiv.org/abs/2306.03310>.
- Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi Jim Fan, and Guanzhi Wang. ASPIRE: Agentic /skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026. <https://arxiv.org/abs/2607.00272>.
- Haitong Ma, Chenxiao Gao, Rushi Qiang, Bo Dai, and Na Li. RLE-Bench: A qualifying exam for coding agents as robot learning engineers. *arXiv preprint arXiv:2609.34210*, 2026. <https://arxiv.org/abs/2609.34210>.
- Jitendra Malik. On LLM robotics demonstrations, dexterity, and dynamics. <https://x.com/JitendraMalikCV/status/2097173961264284039>, 2026. X post, September 8, 2026. Accessed October 4, 2026. Descriptive title.
- Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022. <https://arxiv.org/abs/2112.03227>.
- Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Muñoz, Xinjie Yao, René Zurbrügg, Nikita Rudin, Lukasz Wawrzyniak, Milad Rakhsha, Alain Denzler, Eric Heiden, Ales Borovicka, Ossama Ahmed, Iretiayo Akinola, Abrar Anwar, Mark T. Carlson, Ji Yuan Feng, Animesh Garg, Renato Gasoto, Lionel Gulich, Yijie Guo, M. Gussert, Alex Hansen, Mihir Kulkarni, Chenran Li, Wei Liu, Viktor Makoviychuk, Grzegorz Malczyk, Hammad Mazhar, Masoud Moghani, Adithyavairavan Murali, Michael Noseworthy, Alexander Poddubny, Nathan Ratliff, Welf Rehberg, Clemens Schwarke, Ritvik Singh, James Latham Smith, Bingjie Tang, Ruchik Thaker, Matthew Trepte, Karl Van Wyk, Fangzhou Yu, Alex Millane, Vikram Ramasamy, Remo Steiner, Sangeeta Subramanian, Clemens Volk, CY Chen, Neel Jawale, Ashwin Varghese Kuruttukulam, Michael A. Lin, Ajay Mandlekar, Karsten Patzwaldt, John Welsh, Huihua Zhao, Fatima Anes, Jean-Francois Lafleche, Nicolas Moënné-Loccoz, Soowan Park, Rob Stepinski, Dirk Van Gelder, Chris Amever, Jan Carius, Jumyung Chang, Anka He Chen, Pablo de Heras Ciechowski, Gilles Daviet, Mohammad Mohajerani, Julia von Muralt, Viktor Reutsky, Michael Sauter, Simon Schirm, Eric L. Shi, Pierre Terdiman, Kenny Vilella, Tobias Widmer, Gordon Yeoman, Tiffany Chen, Sergey Grizan, Cathy Li, Lotus Li, Connor Smith, Rafael Wiltz, Kostas Alexis, Yan Chang, David Chu, Linxi "Jim" Fan, Farbod Farshidian, Ankur Handa, Spencer Huang, Marco Hutter, Yashraj Narang, Soha Pouya, Shiwei Sheng, Yuke Zhu, Miles Macklin, Adam Moravanszky, Philipp Reist, Yunrong Guo, David Hoeller, and Gavriel State. Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning. *arXiv preprint arXiv:2511.04831*, 2025. <https://arxiv.org/abs/2511.04831>.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots. In *Robotics: Science and Systems (RSS)*, 2024. <https://github.com/robocasa/robocasa>.
- Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. *arXiv preprint arXiv:2603.04356*, 2026. <https://arxiv.org/abs/2603.04356>.
- NVIDIA. OmniIsaacGymEnvs. Software repository, n.d. <https://github.com/isaac-sim/OmniIsaacGymEnvs>. Accessed 6 October 2026.
- OpenAI. Introducing Operator, 2025. <https://openai.com/index/introducing-operator/>.
- Bikang Pan, Fan Liu, Haotao Lu, Jingya Wang, and Ye Shi. SelfWAM: A Self-Grounded Unified World Action Model for Fast Robot Control. *arXiv preprint arXiv:2608.00725*, 2026. <https://arxiv.org/abs/2608.00725>.
- Zeyu Shen, Haoxiang You, Yilang Liu, Zhicheng Zheng, Lihan Zha, Kashu Yamazaki, Mingtong Zhang, Suning Huang, Jiankai Sun, Qianzhong Chen, Lucy He, Kaiyuan Liu, Haoran Chang, Katerina Fragkiadaki, Dhruv Shah, Mac Schwager, Peter Henderson, Ian Abraham, and Canwen Xu. EmbodiedSWE: Coding agents for long horizon dexterous robotics. *arXiv preprint arXiv:2609.27308*, 2026. <https://arxiv.org/abs/2609.27308>.
- Ruiyang Si, Jianxin Bi, Shunyu Yang, Rui Ni, Wenbo Huang, Qiang Wang, Shulong Jiang, Duomin Wang, Xiuyu Li, Haiwen Feng, Zhen Dong, and Daquan Zhou. Fewer tokens, better action: GPT-6 Astra robot agents with 14% higher success rate but 65% fewer tokens. *arXiv preprint arXiv:2610.01939*, 2026. <https://arxiv.org/abs/2610.01939>.

- Balakumar Sundaralingam, Siva Kumar Sastry Hari, Adam Fishman, Caelan Garrett, Karl Van Wyk, Valts Blukis, Alexander Millane, Helen Oleynikova, Ankur Handa, Fabio Ramos, Nathan Ratliff, and Dieter Fox. cuRobo: Parallelized collision-free minimum-jerk robot motion generation. *arXiv preprint arXiv:2310.17274*, 2023. <https://arxiv.org/abs/2310.17274>.
- Yen-Jen Wang, Haozhe Jiang, Shuying Deng, Haoru Xue, Weirui Ye, Rocky Duan, Nika Haghtalab, S. Shankar Sastry, Pieter Abbeel, and Haozhi Qi. Reconstruct, practice, go real: Guided self-improvement for embodied agents. *arXiv preprint arXiv:2610.02204*, 2026a. <https://arxiv.org/abs/2610.02204>.
- Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An Open, Modular Exploration Towards Systematic World-Action Model Pretraining. *arXiv preprint arXiv:2609.07398*, 2026b. <https://arxiv.org/abs/2609.07398>.
- Botian Xu, Feng Gao, Chao Yu, Ruize Zhang, Yi Wu, and Yu Wang. OmniDrones: An Efficient and Flexible Platform for Reinforcement Learning in Drone Control, 2023. <https://arxiv.org/abs/2309.12825>.
- Zelai Xu, Ruize Zhang, Chao Yu, Huining Yuan, Xiangmin Yi, Shilong Ji, Chuqi Wang, Wenhao Tang, Feng Gao, Wenbo Ding, Xinlei Chen, and Yu Wang. VolleyBots: A Testbed for Multi-Drone Volleyball Game Combining Motion Control and Strategic Play, 2025. <https://arxiv.org/abs/2502.01932>.
- Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, Heng Ji, Huan Zhang, and Tong Zhang. EmbodiedBench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. *arXiv preprint arXiv:2502.09560*, 2025. <https://arxiv.org/abs/2502.09560>.
- Xuning Yang, Rishit Dagli, Alex Zook, Hugo Hadfield, Ankit Goyal, Stan Birchfield, Fabio Ramos, and Jonathan Tremblay. RoboLab: A High-Fidelity Simulation Benchmark for Analysis of Task Generalist Policies. In *Proceedings of Robotics: Science and Systems*, Sydney, Australia, July 2026a. <https://arxiv.org/abs/2604.09860>.
- Zhenjie Yang, Yideng Zhang, Dongjie Zhang, Chenyu Jiang, Xianshuai Liu, Yufeng Li, Zuhao Ge, Xingyu Jiao, Zheng Zhang, Kaiyu He, He Wang, Yuwen Zhong, Yi Deng, Muyun Jiang, Xianliang Huang, Haisheng Su, Donghang Zhang, Jian Zhang, Xue Yang, Hongyang Li, Zuxuan Wu, Yu-Gang Jiang, Xiaosong Jia, and Junchi Yan. Bench2Dex: Benchmarking Visuo-Tactile Bimanual Dexterous Manipulation Across Dexterous Hands. *arXiv preprint arXiv:2609.15726*, 2026b. <https://arxiv.org/abs/2609.15726>.
- Shiduo Zhang, Zhe Xu, Peiju Liu, Xiaopeng Yu, Yuan Li, Qinghui Gao, Zhaoye Fei, Zhangyue Yin, Zuxuan Wu, Yu-Gang Jiang, and Xipeng Qiu. VLABench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. *arXiv preprint arXiv:2412.18194*, 2024. <https://arxiv.org/abs/2412.18194>.
- Yinghua Zhou, Junjie Ye, Yiqi Zhao, Hao Dong, Celina Shiyu Wang, Ruohai Ge, Tingyi Yang, Basile Van Hoorick, Gaurav Sukhatme, Vitor Guizilini, and Yue Wang. Rolling-WAM: World Action Models with Rolling Imagination. *arXiv preprint arXiv:2609.30247*, 2026. <https://arxiv.org/abs/2609.30247>.
- Zhehua Zhou, Jiayang Song, Xuan Xie, Zhan Shu, Lei Ma, Dikai Liu, Jianxiong Yin, and Simon See. Towards Building AI-CPS with NVIDIA Isaac Sim: An Industrial Benchmark and Case Study for Robotics Manipulation. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, 2024. doi: 10.1145/3639477.3639740. <https://arxiv.org/abs/2308.00055>.
- zyicome. Wheel-Legged-Lab. Software repository, n.d. <https://github.com/zyicome/Wheel-Legged-Lab>. Accessed 6 October 2026.

A Benchmark Specification and Task Profiles

A.1 Task inventory and reading guide

This appendix describes the retained evaluation snapshot dated 6 October 2026: 84 tasks from 20 source groups, with five scored model records per task. VolleyBots 1v1 is included. Source groups record software provenance; they are not counts of distinct robot embodiments. The five primary domains contain 38 manipulation tasks, 20 mobile manipulation tasks, 11 locomotion tasks, 11 driving tasks, and 4 aerial tasks. Mobile manipulation comprises the ten RoboCasa and ten BEHAVIOR-1K tasks; their mobile base and arm interfaces distinguish them from fixed-base manipulation. Locomotion includes legged motion, whole-body balance and body-coordinated interaction.

The eight profiles below connect each task objective to what the model can sense, what its tools command, and what the evaluator checks. Each profile pairs actual recorded views with a concise interface specification. The complete task inventory appears in Appendix D; the registered tool catalog and original prompt examples appear in Appendices B and C. Task objectives here are condensed summaries; quoted prompt text is identified separately. The inventory credits each originating project. Paper citations identify the corresponding published work or preprint; software citations identify projects for which no separate project paper was found. Digit uses the Isaac Lab environment, TTRL uses the PACE project, and the VolleyBots source includes the JuggleRL task.

Task source	Tasks	Reference
HumanoidSoccer	1	(Kong et al., 2026)
ReflexBench	1	(Chen et al., 2026c)
Digit	1	(Mittal et al., 2025)
Flamingo	1	(jaykorea, n.d.); software
Go2 Push	1	(BrandoUlissi, n.d.); software
AI-CPS	3	(Zhou et al., 2024)
OmniIsaacGymEnvs	1	(NVIDIA, n.d.); software
OmniDrones	2	(Xu et al., 2023)
Bench2Dex	9	(Yang et al., 2026b)
RoboCasa	10	(Nasiriany et al., 2024, 2026)
RoboLab	10	(Yang et al., 2026a)
BEHAVIOR-1K	10	(Li et al., 2024a)
WheeledLab	11	(Han et al., 2025)
Robot Lab	1	(Fan, 2024); software
SteadyTray	1	(Huang et al., 2026)
TTRL	1	(Hu et al., 2026)
VolleyBots	2	(Xu et al., 2025; Ji et al., 2025)
Wheel-Legged	2	(zyicome, n.d.); software
Wheeled Quadruped	1	(Asfaw, 2026); software
RoboDojo	15	(Chen et al., 2026a)

A.2 Primary task types

The primary types below form an editorial partition based on the task objective. A grasp, contact event, or intermediate motion can occur in several types; these counts are task counts, not frequencies of atomic actions in trajectories. Secondary demands and success-check families can overlap.

Domain	Primary task type	Tasks
Manipulation	Fitting & insertion	7
Manipulation	Placement & organisation	13
Manipulation	Pouring & processing	3
Manipulation	Multi-stage workflows	8
Manipulation	Articulation & device use	3
Manipulation	Dynamic manipulation	4
Mobile manipulation	Navigation & workflows	8
Mobile manipulation	Pouring & processing	2
Mobile manipulation	Placement & organisation	8
Mobile manipulation	Articulation & device use	2
Locomotion	Carrying & ball interaction	3
Locomotion	Locomotion & terrain	4
Locomotion	Balance & recovery	4
Driving	Drift & timed passage	4
Driving	Routes & terrain	5
Driving	Precision parking	2
Aerial	Stabilisation & tracking	2
Aerial	Ball interaction	2

Manipulation (38 tasks). Fixed-base arms and dexterous hands establish object relations or operate objects within a workspace. Placement and organisation includes sorting, stacking and reorientation. Fitting and insertion includes chargers, tubes and supported placements such as hanging a mug. Pouring and processing is distinguished by the material-transfer objective. A multi-stage workflow combines distinct goals, such as storing multiple items or using a device; routine approach, grasp and lift motions alone do not create this category. Dynamic manipulation involves a moving target or sustained object regulation.

Mobile manipulation (20 tasks). The ten RoboCasa and ten BEHAVIOR-1K tasks expose a mobile base together with manipulation tools. The domain describes the available embodiment and control problem; it does not require every successful episode to move the base. For example, drawer closure remains a mobile-manipulation task even when the arm performs the decisive final motion. Primary task types still describe the objective: placement, articulation, processing or navigation and household workflows. This separates mobility from the object operation being evaluated.

Locomotion (11 tasks). This domain includes legged movement, terrain traversal, posture recovery and whole-body coordination. Balance and recovery tasks emphasise the body state rather than distance travelled. Carrying and ball interaction covers tasks where motion of the body must be coordinated with a load or a ball, such as tray transport and humanoid ball interaction. Wheel-legged platforms remain in this domain when their task tests posture or legged mobility rather than vehicle steering along a route.

Driving (11 tasks). Vehicle tasks use speed and steering to traverse routes, negotiate terrain, drift or park. Drift and timed passage is separated from ordinary route following because a transient manoeuvre or moving constraint is central to the objective. Precision parking instead emphasises the terminal pose and alignment. A driving trajectory can contain all three behaviours while still contributing only once to the inventory under its primary objective.

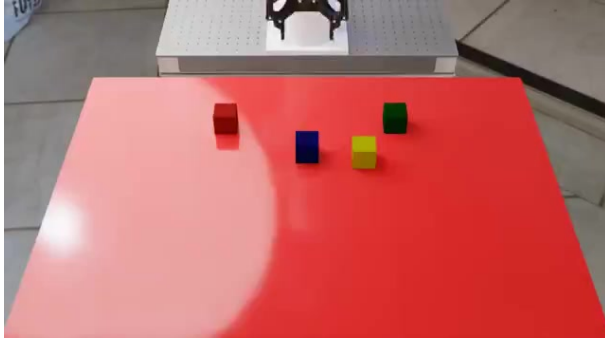
Aerial (4 tasks). Two tasks test payload stabilisation or inverted-pendulum tracking, and two test drone–ball interaction. The former require sustained regulation of multiple state variables; the latter also depend on contact histories and timing. Across all five domains, the finer task types are descriptive categories, while the executable checker remains the authority for success. Secondary requirements such as constrained contact or multiple subgoals can overlap and are analysed separately in Section 6.

A.3 Task profile 01: Ordered block stacking

RoboLab / Placement and organisation

Fixed-base DROID: Franka arm and Robotiq gripper

Objective. Stack the blocks from bottom to top in the order red, blue, green, yellow.



Video 0.10 s



Video 40.98 s

Astra; external review camera. The late frame precedes the terminal gripper request.

01 / SENSE

RGB camera views and measured arm, gripper and end-effector state. Absolute IK targets use robot-root metres and WXYZ orientation. The commanded point is the gripper flange, not the fingertips.

02 / ACT

`move_eef`, `set_gripper`, and coordinated robot commands. Ordinary segments request 1–30 steps at 15 Hz. Opening is `gripper_close=0`; closing is 1.

03 / VERIFY

The native stacked relation checks the complete red–blue–green–yellow order. The audited source uses a default 1 cm relation tolerance. An executed approach or a visually plausible partial stack does not establish completion.

EPISODE

1,350 control steps (90 simulated seconds). This retained run ends successfully at result step 635; the last completed event is step 634.

04 / EXECUTION

The ordered tower requires repeated object localisation, grasping, lifting, placement and release. Progress on one block must survive later approaches: neither reaching a target pose nor closing the gripper establishes that the intended block is being carried. The observation after release is needed to distinguish a supported placement from a block still held by the robot.

05 / TRAJECTORY

In the retained Astra episode, the gripper closes at step 157 and the block is lifted by step 179. Camera fitting first appears at step 204, after the initial grasp, and the first placement-and-retraction milestone is step 270. The complete ordered-stack result is successful at step 635. These milestones illustrate a visual estimate followed by geometric refinement, rather than calibration before every motion.

Retained outcomes. Astra succeeds, Opus 5.5 fails, Kimi K3 fails, DeepSeek V4.1 Flash fails, Gemini 3.8 Flash fails. One episode per model.

Reading the episode

Pose arrival, object retention, support and release are separate facts. Preserve clearance for the whole carried block.

Record: astra-62, BlockStackingSpecifiedOrderTask. Recorded views and outcomes refer to this retained episode.

A.4 Task profile 02: Plugging in a charger

RoboDojo / Fitting and insertion

Two fixed ARX X5 arms with parallel-jaw grippers

Objective. Plug the charger into the power strip.



Video 0.10 s



Video 12.61 s

Astra; recorded head-camera view.

01 / SENSE	RGB views plus both grasp-point poses and contextual joint angles. Named Cartesian targets are absolute world coordinates; wrist-angle parameters use degrees relative to a downward reference.
02 / ACT	<code>move_eef</code> changes named pose and gripper fields. Unnamed fields hold their measured call-start values. Opening is 1; closing is 0. The planner chooses trajectory length; the caller does not specify a step count.
03 / VERIFY	The audited native task requires the charger in the socket region, a 1.5 cm depth threshold and its specified axis aligned upward within 10 degrees. The robot must also satisfy the return-to-home condition.
EPISODE	400 control steps. The retained Astra episode succeeds at step 323 after 42 robot requests.
04 / EXECUTION	The control sequence must establish a grasp, transport the charger, orient it relative to the power strip, insert it and return the robot to its required final configuration. Because both arms occupy the workspace, a feasible target for one end effector may still produce a rejected motion when the other arm obstructs the trajectory.
05 / TRAJECTORY	The retained Astra trajectory contains a rejected rotation at call 17, with the step counter at 196. The next call lowers the idle arm, reaching step 207. Retrying the rotation then succeeds at step 230; the episode subsequently passes the insertion and return conditions at step 323. The feedback therefore changes the configuration before the same intended rotation is attempted again.

Retained outcomes. Astra succeeds, Opus 5.5 fails, Kimi K3 fails, DeepSeek V4.1 Flash fails, Gemini 3.8 Flash fails. One episode per model.

Reading the episode

Lateral alignment alone is insufficient: insertion depth, orientation and robot return state also matter.

Record: task-338, plug_in_charger. Recorded views and outcomes refer to this retained episode.

A.5 Task profile 03: Closing a kitchen drawer

RoboCasa / Articulation and device use

RoboCasa PandaOmron mobile manipulator

Objective. Close the designated drawer.



Video 0.10 s



Video 12.93 s

Kimi K3; recorded multi-camera strip.

01 / SENSE	RGB and robot proprioception. Reported end-effector pose is base-relative, with XYZW quaternions. The active action is a normalised 12-dimensional controller vector.
02 / ACT	Arm commands use OSC increments, with translation up to 0.05 m and rotation up to 0.5 rad per repeated step. Repeating a nonzero increment repeats motion. Base and torso commands are normalised inputs.
03 / VERIFY	All door joints belonging to the designated drawer must have normalised opening at most 0.05. The checker evaluates the articulated state; reaching the handle is an intermediate action.
EPISODE	450 control steps. Kimi K3 completes this retained episode at step 265 using 19 robot requests.
04 / EXECUTION	A mobile manipulator must establish a usable base-to-drawer relation before the arm can apply a closing motion. End-effector commands are increments in the robot base frame, so their effect depends on both the current arm pose and the duration of repetition. Small segments near the drawer allow fresh images to reveal progress or unintended contact.
05 / TRAJECTORY	Kimi K3 shortens its five late pushing requests to 20, 20, 12, 6 and 5 steps. The subsequent withdrawal asks for 12 steps, but the environment executes only two before reporting success at step 265. The shortened final request reflects episode termination by the native checker. It does not mean that the full requested retreat was necessary for the drawer to count as closed.

Retained outcomes. Astra succeeds, Opus 5.5 fails, Kimi K3 succeeds, DeepSeek V4.1 Flash fails, Gemini 3.8 Flash fails. One episode per model.

Reading the episode

The RoboCasa incremental interface differs from RoboLab absolute IK despite similarly named tools.

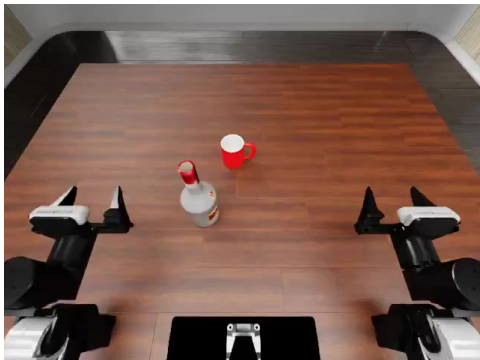
Record: task-7, CloseDrawer. Recorded views and outcomes refer to this retained episode.

A.6 Task profile 04: Pouring into a cup

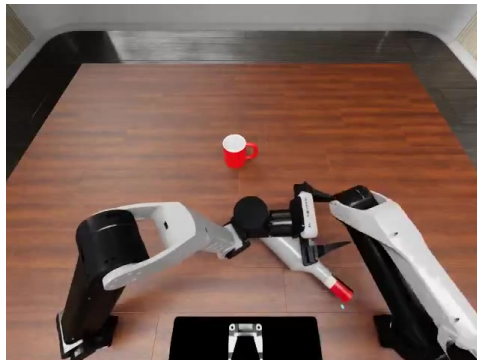
RoboDojo / Pouring and processing

Two fixed ARX X5 arms with parallel-jaw grippers

Objective. Pour liquid from the bottle into the cup.



Video 0.10 s



Video 15.52 s

Kimi K3; recorded head-camera view; unsuccessful episode.

01 / SENSE

The same world-frame grasp-point interface as charger insertion. RGB observations identify the bottle and cup; no privileged object-coordinate query is supplied.

02 / ACT

`move_eef` requests bounded planned motion. Gripper change follows arm arrival in a combined request. The tool reports arrival residuals and planner failures.

03 / VERIFY

The audited native liquid test is triggered when the bottle becomes upright again within 30 degrees. It uses a cup-transfer threshold of 0.97 and bottle-residual threshold of 0.15 under the original fluid-filtering rule.

EPISODE

400 control steps. The illustrated Kimi K3 run consumes the horizon without completing the task; 27 robot requests are recorded.

04 / EXECUTION

Pouring couples object retention, bottle orientation and the relative position of the receiving cup. A successful tool return establishes the executed arm motion, but the next observation must establish whether the bottle remains in the gripper and whether its opening is above the cup. The liquid test is separate from the arm-arrival residual.

05 / TRAJECTORY

In this Kimi K3 episode, the approach topples the bottle at step 42. Repeated left-arm attempts fail to establish a usable grasp; the agent changes strategy at step 240 and repositions by step 272. Its right-arm attempt remains unfinished when the 400-step horizon is exhausted. The example exposes a delay between losing the intended object state and making a substantial change to the approach.

Retained outcomes. Astra fails, Opus 5.5 fails, Kimi K3 fails, DeepSeek V4.1 Flash fails, Gemini 3.8 Flash fails. One episode per model.

Reading the episode

Moving a grasp point to the requested pose does not show that a bottle was grasped or that liquid entered the cup.

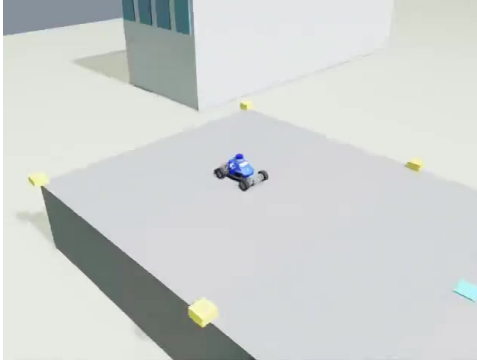
Record: task-25, pour_liquid_into_cup. Recorded views and outcomes refer to this retained episode.

A.7 Task profile 05: Following a courtyard route

WheeledLab / Routes and terrain

MuSHR wheeled vehicle

Objective. Follow the road to the green parking patch, align with the direction of travel, and stop.



Video 0.10 s



Video 36.54 s

Kimi K3; external reviewer camera, not the policy front camera.

01 / SENSE	The policy receives a front RGB camera, wheel and steering encoders, and body IMU attitude/gyro. Global pose, a full map and checkpoint progress are not policy inputs.
02 / ACT	drive supplies normalised speed and steering. In this MuSHR profile, negative speed clamps to zero; there is no reverse. A held segment spans 1–50 control steps at 50 Hz.
03 / VERIFY	For this archived contract, complete the ordered route and reach the destination within 0.65 m, heading within 25 degrees and ground speed at most 0.12 m/s for 0.8 s. Road and collision conditions remain active.
EPISODE	2,000 control steps (40 simulated seconds). Kimi K3 succeeds at step 1,881 with 6/6 checkpoints and a recorded 0.8 s parking hold.
04 / EXECUTION	The route requires steering through ordered checkpoints before satisfying the terminal parking condition. Steering and speed must be inferred from the front-camera scene and local sensors: the external video is useful for reviewing the episode, but it does not supply the agent with a map or global route progress. Shortening a held command creates another opportunity to observe before a turn.
05 / TRAJECTORY	The retained Kimi K3 result reaches all six checkpoints and records a 0.8-second parking hold, succeeding at step 1,881 of 2,000. Reaching the green region alone would leave heading, speed and dwell requirements unresolved. Conversely, a low final speed away from the required destination would not count as route completion. The trajectory must satisfy the route and terminal checks together.

Retained outcomes. Astra succeeds, Opus 5.5 succeeds, Kimi K3 succeeds, DeepSeek V4.1 Flash fails, Gemini 3.8 Flash fails. One episode per model.

Reading the episode

The dwell interval is specific to this recorded version. Do not substitute a newer arrival-only checker for a historical trajectory.

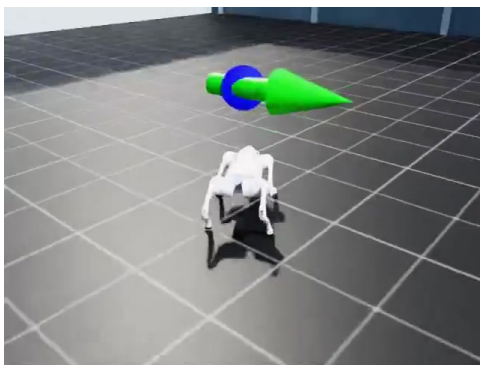
Record: task-107, rw-courtyard. Recorded views and outcomes refer to this retained episode.

A.8 Task profile 06: Quadruped disturbance recovery

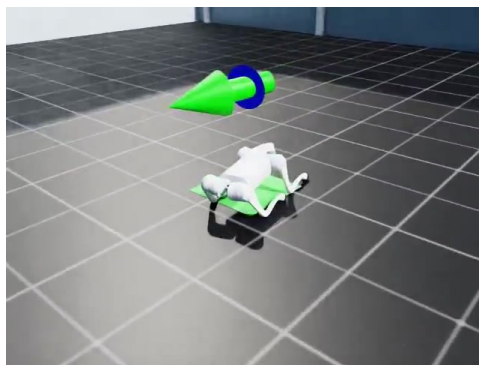
Go2 Push / Balance and recovery

Go2 quadruped, 12 joint-position residuals

Objective. Recover near the prescribed root goal while remaining upright and slowing down under the retained scene dynamics.



Video 0.10 s



Video 19.40 s

Opus 5.5; third-person reviewer video.

01 / SENSE

Named native actor terms include body velocities, projected gravity, commands, joint state and previous action. These are simulator-derived actor observations, not an RGB-only setting.

02 / ACT

`apply_action` supplies 12 simultaneous residuals. Joint targets follow $q_{\text{target}} = q_{\text{default}} + 0.25u$ radians. Native motor PD remains active; no learned gait controller is supplied.

03 / VERIFY

The recorded World-state checker requires, throughout the final 2 s, horizontal goal error at most 0.25 m, horizontal speed at most 0.10 m/s and tilt at most 20 degrees. Native or scene failure overrides success.

EPISODE

1,000 control steps at 50 Hz (20 simulated seconds). Opus 5.5 completes the full window; final error is 0.192 m, speed 0.0022 m/s and tilt 4.48 degrees.

04 / EXECUTION

Twelve residual joint targets are applied together, so posture regulation requires coordinating the entire stance. The previous action and measured body motion provide feedback for the next bounded request. The native PD controller converts targets into motor responses; it does not choose a learned recovery policy for the agent.

05 / TRAJECTORY

The retained Opus 5.5 episode completes the 1,000-step horizon. Its terminal horizontal error, speed and tilt are below their individual thresholds, but these endpoint values alone are not the success test. The checker also requires the conjunction to hold throughout the final two seconds. This separates sustained recovery from briefly crossing a favourable pose during continuing motion.

Retained outcomes. Astra fails, Opus 5.5 succeeds, Kimi K3 fails, DeepSeek V4.1 Flash fails, Gemini 3.8 Flash fails. One episode per model.

Reading the episode

The binary World-state criterion is distinct from the upstream velocity-tracking reward. The task name alone does not measure recovery from every possible disturbance.

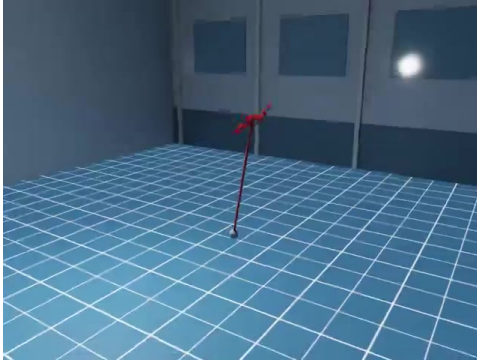
Record: task-142, quadruped_push_recovery. Recorded views and outcomes refer to this retained episode.

A.9 Task profile 07: Stabilising a suspended payload

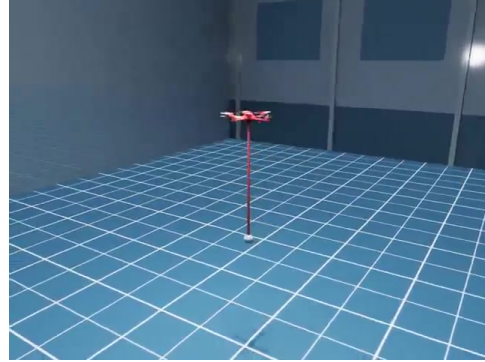
OmniDrones / Stabilisation and tracking

Quadrotor with a 1 m suspended payload

Objective. Stabilise the payload at its target and suppress swing under native random payload pushes.



Video 0.10 s



Video 7.84 s

Opus 5.5; external reviewer video.

01 / SENSE

The native actor vector exposes relative payload/target geometry, drone attitude and motion, and declared motor-state terms. No hidden evaluator state is returned.

02 / ACT

`apply_action` supplies four rotor inputs. Desired internal throttle is $s^* = \sqrt{\text{clip}((u+1)/2, 0, 1)}$; thrust depends on s^2 . Motor lag remains active.

03 / VERIFY

For the final 2 s: payload error at most 0.15 m, payload speed at most 0.20 m/s, suspension swing at most 10 degrees, and drone tilt at most 15 degrees. All bounds must hold together through the end.

EPISODE

500 control steps at 62 Hz (8.0645 simulated seconds). Opus 5.5 passes with final payload error 0.0394 m and speed 0.0147 m/s.

04 / EXECUTION

The controller must regulate both the quadrotor and its suspended load. Changing thrust can correct altitude while exciting swing, and motor lag separates the requested rotor input from its immediate physical effect. Feedback therefore needs to track payload displacement and velocity alongside drone tilt, rather than treating an upright drone as sufficient evidence of success.

05 / TRAJECTORY

The Opus 5.5 trajectory finishes with payload error 0.0394 m and speed 0.0147 m/s. The recorded joint-stability interval lasts 2.15 seconds, exceeding the required two seconds, with all four bounds satisfied together. The episode illustrates maintaining a recovered state until the end instead of stopping at the first apparently stable observation.

Retained outcomes. Astra fails, Opus 5.5 succeeds, Kimi K3 fails, DeepSeek V4.1 Flash fails, Gemini 3.8 Flash fails. One episode per model.

Reading the episode

Stable drone attitude does not imply a stable payload. A single passing frame does not satisfy the temporal window.

Record: task-144, drone_payload_hover. Recorded views and outcomes refer to this retained episode.

A.10 Task profile 08: Repeated drone ball juggling

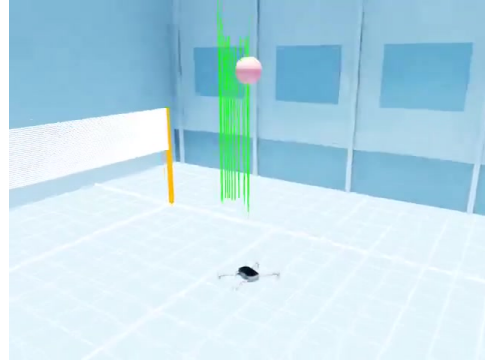
VolleyBots / Ball interaction

Iris quadrotor with native body-contact dynamics

Objective. Keep the ball airborne through repeated legal contacts and sufficiently high flight arcs.



Video 0.10 s



Video 15.52 s

Opus 5.5; external reviewer video, unavailable to the policy.

01 / SENSE

A 32-value native actor vector supplies drone pose/motion, anchor and ball-relative terms, ball velocity and episode time. The review camera is not part of the actor input.

02 / ACT

Four rotor commands through `apply_action`, held for bounded segments at 50 Hz. No automatic ball-launch or racket assistance is added. Thinking pauses simulator time.

03 / VERIFY

At least four height-qualified hits are required. A qualifying arc exceeds ball-centre height 3.5 m and is confirmed at the next valid contact. Contacts separated by at most 25 steps are invalid. The final 0.6 s must keep the ball airborne without support contact.

EPISODE

800 control steps (16 simulated seconds). Opus 5.5 records 15 true hits, 14 height-qualified hits and 274 robot requests, completing the full horizon.

04 / EXECUTION

The agent alternates phases that bring the drone into contact with the ball and phases that prepare for its next return. Commands expose rotor inputs rather than a high-level strike primitive. Ball position and velocity must be interpreted with the drone state; a rendered near-contact frame cannot establish a legal hit or reconstruct the intervening flight arc.

05 / TRAJECTORY

Opus 5.5 issues 274 robot requests across the 800-step horizon, with a median three executed steps per request. The checker records 15 valid hits and 14 height-qualified hits. The trajectory also satisfies the final airborne interval. These three facts are distinct: frequent calls alone do not ensure valid contacts, and enough historical hits do not excuse support contact in the final window.

Retained outcomes. Astra fails, Opus 5.5 succeeds, Kimi K3 fails, DeepSeek V4.1 Flash fails, Gemini 3.8 Flash fails. One episode per model.

Reading the episode

Visible proximity is not a contact event. The final position alone cannot reconstruct the legal-hit sequence.

Record: task-158, drone_volleyball_solo_juggle. Recorded views and outcomes refer to this retained episode.

B Execution, Observations, and Evaluation Records

B.1 What the archive contains

The offline archive contains one self-contained task page for each of the 420 retained model-task records. Each page carries a manifest entry and embedded run, tool and environment records, a merged event trace, and any available videos. We verified the SHA-256 of all 420 pages against the five-model result audit before extracting this appendix. The frozen snapshot is `snapshot-20261006T025143Z`. The publication-side inventory maps each task to its source group, primary domain, model, local trace ID, outcome and page hash.

The data support three distinct readings. The manifest supplies the *retained scored outcome*; the embedded run preserves the *historical terminal result*; and the event/video stream records the *observed execution*. These may differ after budget adjudication. Appendix H gives a concrete example in which the historical terminal success occurs after the scored boundary. We never replace the retained score with the last visually plausible frame.

B.2 Registered robot tools across the benchmark

The following catalog is extracted from the startup tool schemas of all 420 retained records. It describes tools offered to the model, including tools that a particular episode never calls. Tool-name sets agree across the five models within each source group. Parameters and action maps can still vary by task. All 20 source groups are covered: the 13 native-action groups are expanded in the action-map table below.

Source group	Tasks	Registered robot tools
RoboDojo	15	<code>move_eef</code>
RoboLab	10	<code>move_eef</code> , <code>set_gripper</code> , <code>move_robot</code>
RoboCasa	10	<code>move_eef</code> , <code>move_base</code> , <code>move_torso</code> , <code>set_gripper</code> , <code>move_robot</code>
BEHAVIOR-1K	10	<code>move_arms</code> , <code>move_base</code> , <code>set_grippers</code> , <code>move_torso</code> , <code>move_robot</code>
WheeledLab	11	<code>observe</code> , <code>drive</code>
AI-CPS	3	<code>observe</code> , <code>move_joints</code>
HumanoidSoccer	1	<code>move_joints</code>
Native-action adapters (13 source groups)	24	<code>observe</code> , <code>apply_action</code>

Robot tools and workspace tools. Shell execution and image-file viewing belong to the frontier harness, not to the robot-tool catalog. They support calculations, inspection of supplied observations and notes; they do not advance physics. An `observe` call also advances no control steps, but remains subject to the interaction budget. Adapters without a standalone `observe` tool return observations through motion feedback.

Effective availability. No retained startup registers `coding_control`, `reset`, `score-query` or `give-up` tools. Some inherited prompt passages describe optional coding control; the registered tool list and disabled run setting determine what was actually available. The source supplement `appendix_interfaces.json` stores the original instructions and schemas, linked to each run and source-page hash.

B.3 Manipulation tools: arguments and execution meaning

Most motion tools take a short `note`, a duration `steps`, and a `targets` object. RoboDojo is an exception: its planner chooses the trajectory duration, so the exposed call has no `steps` argument. End-effector (EEF) coordinates and gripper polarity differ across adapters.

Adapter	Tool	Inputs and meaning
RoboDojo	<code>move_eef</code>	<code>targets</code> : selected left/right XYZ (world metres), wrist pitch/roll/yaw (degrees from the downward reference), and gripper opening (0 closed, 1 open). Omitted fields hold measured call-start values. The planner moves the arm before applying a combined gripper change.
RoboLab	<code>move_eef</code>	<code>targets.position[3]</code> and/or <code>quaternion_wxyz[4]</code> : absolute flange pose in the robot-root frame. Omitted pose components hold measured values. <code>steps=1..30</code> at 15 Hz.
RoboLab	<code>set_gripper</code>	<code>targets.gripper_close</code> : 0 opens, 1 closes; the target persists. The arm holds while the segment executes.
RoboLab	<code>move_robot</code>	Combines the same arm-pose and gripper fields in one native action. Closure starts with arm motion, not after arrival.
RoboCasa	<code>move_eef</code>	<code>targets.eef_delta[6]</code> : normalised translation and rotation-vector increments in the base frame. Scales are 0.05 m and 0.5 rad per step. <code>steps=1..30</code> ; repetition repeats the increment.
RoboCasa	<code>move_base</code>	<code>targets.base_motion[3]</code> : normalised XY/yaw controller inputs, not displacement. Uses base-following mode; gripper target persists.
RoboCasa	<code>move_torso</code>	<code>targets.torso</code> : normalised input in [-1,1]; requests a slide-joint increment of 0.05 times the input in metres per tick. Not an absolute torso height.
RoboCasa	<code>set_gripper</code>	<code>targets.gripper_close</code> : 0 opens, 1 closes. No arm/base/torso motion is requested.
RoboCasa	<code>move_robot</code>	Combines EEF, base, torso and gripper fields. Optional <code>control_mode=0</code> updates from achieved arm pose; 1 uses the desired pose for base following.

Same name, different motion. Holding a RoboLab absolute pose repeats a fixed target. Holding a RoboCasa nonzero EEF delta requests an increment every step. A RoboDojo target addresses the grasp point between the jaws; RoboLab addresses the flange. These distinctions determine whether a repeated call holds, accumulates motion, or changes the physical point being controlled.

B.4 Mobile manipulation, driving and named joints

Adapter	Tool	Inputs and meaning
BEHAVIOR-1K	<code>move_arms</code>	Left/right XYZ in robot-root metres and <code>left/right_quat_xyzw</code> . Absolute EEf IK; omitted arm holds its measured pose. All tools in this group use <code>steps=1..30</code> at 30 Hz.
BEHAVIOR-1K	<code>move_base</code>	<code>base_vx</code> , <code>base_vy</code> in local-body m/s (limits +/-0.3), and <code>base_wz</code> in rad/s (+/-0.5). Arms hold root-relative targets; omitted base velocities are zero.
BEHAVIOR-1K	<code>set_grippers</code>	<code>left/right_gripper</code> : continuous opening in [0,1], with 0 closed and 1 open. Targets persist; EEfs hold measured poses and base motion stops.
BEHAVIOR-1K	<code>move_torso</code>	<code>trunk_qpos[4]</code> : absolute trunk joint positions in radians, using the recorded joint order and limits. This differs from the normalised slide increment in RoboCasa.
BEHAVIOR-1K	<code>move_robot</code>	Combines arm poses, gripper openings, base velocities and trunk targets simultaneously. One control step counts once, regardless of how many components are commanded.
WheeledLab	<code>observe</code>	Returns the current allowed observation without stepping physics. Authored onboard courses expose front RGB, encoders and IMU; the call does not reveal map, world pose or checkpoint progress.
WheeledLab	<code>drive</code>	<code>action[2]</code> = normalised speed and steering, each in [-1,1]; <code>steps=1..50</code> . Speed scales to 3 m/s wheel targets; steering uses 0.488 followed by the native mapping. Reverse availability is task-specific.
AI-CPS	<code>observe; move_joints</code>	Observation costs no physics steps. Motion uses <code>arm_action[7]</code> in [-1,1] for 1–50 steps. Each tick adds 0.125 times the input in radians to the previous target, before limits and native noise; fingers remain task-controlled.
HumanoidSoccer	<code>move_joints</code>	<code>joint_positions</code> : a dictionary of named G1 joint targets in absolute radians; omitted joints hold measured call-start positions. <code>steps=1..50</code> at 50 Hz. Targets pass through the original PD torque controller. If the recorded ankle-balance assist is enabled, it can adjust ankle targets; it is not a walking policy.

Profile-specific details. The archived courtyard profile clamps negative speed to zero. Reverse-bay and parallel-parking profiles accept signed reverse motion. Likewise, `move_joints` names absolute joint positions in HumanoidSoccer but repeated normalised increments in AI-CPS. The supplied action contract, not the tool name alone, determines the mapping.

B.5 Native-vector tools and action maps

All 13 source groups below register `observe(note)` and `apply_action(note, action, steps)`. Observation returns the current native actor-policy input without stepping physics. Execution supplies the entire vector simultaneously for 1–50 control steps. Native gains, lags, constraints and per-task termination remain active. The vector length is validated against the archived schema; the entries are not universally joint angles or bounded to $[-1,1]$.

Source group	Dim.	Meaning of the action vector
ReflexBench	8	Absolute root-frame EEF XYZ + WXYZ quaternion + gripper sign. Positive gripper opens; non-positive closes.
Digit	26	Joint-position targets with the recorded offsets and 0.5 rad input scale, in runtime joint order.
Flamingo	8	Six actuator-position channels and two wheel-velocity channels (40 rad/s scale). Leg channels use motor-space angles with gear ratio -1.5.
Go2 Push	12	Joint-position residuals: $q_{\text{target}} = q_{\text{default}} + 0.25u$ radians.
OmniIsaacGymEnvs	12	ANYmal joint-position offsets: $q_{\text{target}} = q_{\text{default}} + 0.5u$ radians, tracked by native PD.
OmniDrones	4	Four rotor inputs. -1 requests zero thrust; +1 requests maximum. Zero is half maximum steady-state thrust, not hover.
Bench2Dex	52	Absolute arm and dexterous-finger joint targets in radians. Both UR5 arms and both hands act simultaneously; no binary gripper command.
Robot Lab	12	A1 joint-position residuals with the recorded default posture and 0.25 rad scale; native target clipping remains active.
SteadyTray	29	G1 joint-position targets from per-joint scales plus the default posture. The delayed native actuators remain active.
TTRL	21	Booster T1 joint-position residuals with 0.25 rad scale plus the default posture; raw inputs are clipped to $[-100,100]$.
VolleyBots	4	Four Iris rotor inputs with the native motor mapping and lag; no automatic attitude or ball-tracking policy.
Wheel-Legged	6	Virtual leg angle, leg length and wheel speed for each side. Scales: 0.35 rad, 0.06 m around 0.237 m, and 24 rad/s; native VMC maps them to motor torques.
Wheeled Quadruped	4	Two front-thigh position residuals (0.5 rad scale) and two rear-wheel velocity targets (5 rad/s scale).

What feedback establishes. A valid request can still produce tracking error, collision, a lost grasp or task failure. Conversely, an invalid request can consume an interaction without advancing physics. Inspect the validation receipt, completed-step count and returned observation separately from the evaluator’s task outcome.

B.6 Control-step and interaction accounting

Physics advances through accepted robot execution and is paused during model deliberation and offline calculations. A requested segment can terminate early, be rejected, or finish without meeting the task goal. We therefore distinguish requested steps, confirmed executed steps and the final evaluator’s step count. The Astra stacking record, for example, has a retained result at step 635 and completed environment events through step 634; its terminal feedback is unavailable. This one-step discrepancy is preserved rather than silently repaired.

All 420 run records report environment-side code control disabled. Shell analysis is a separate permission; the retained RoboDojo comparison uses Shell-on. Some inherited prompt text describes an optional coding interface. Its mention does not override the effective run setting or demonstrate that a coding-control call was executed.

The scored protocol uses task-specific physical horizons. For 83 tasks the consecutive non-action limit is 15, with total limits of 30, 60 or 120 according to the task. Volleyball 1v1 uses 20 consecutive and 150 total. Auxiliary calls, zero-step robot requests, and otherwise action-free completed turns are counted under the observable-event policy. Confirmed control execution resets the consecutive counter only. API retries are excluded. The logs mark accounting as incomplete where built-in calls, terminal polling, or turn-end identifiers are not fully observable; an absent trace item is not automatically zero resource use.

B.7 Success checks and their observation window

Sixty-eight tasks per model use a retained **native** scoring profile and sixteen use **world-state-v1**. “Native” is a record label: for authored driving scenarios it includes the versioned independent geometry/trajectory evaluator. Native numerical rewards remain diagnostics and are not averaged across heterogeneous tasks. The World-state records expose whether the full evaluation horizon completed, whether the scene was valid, the sampled predicates, and any overriding failures.

For temporal checks, the claim is limited to the logged control ticks, including reset when recorded. It is not a guarantee over unobserved continuous physics. In payload hovering, all four bounds must hold throughout the final two seconds. In juggling, both the event count and the final airborne window matter. The cards in Appendix A show the thresholds used in these retained examples.

B.8 Frame provenance

The selected task prompts document current observations and up to four historical observations sampled every two feedback rounds, with step labels. Observation rounds have variable physical duration. The environment state supplied to the agent, the history retained by the host, and the continuous reviewer video are separate records.

Every appendix frame is extracted from an embedded source video with a recorded video timestamp and SHA-256. Frames are not generated illustrations. A video timestamp denotes playback time in that file, not elapsed model time or a guaranteed one-to-one correspondence with a tool event. External driving and aerial cameras are reviewer views, not additional model observations. Stage captions identify trace-event indices separately, and late frames can precede terminal success. The accompanying `appendix_frames.json` and `appendix_excerpts.json` resolve these references to source files.

C Prompts and Agent Instructions

System instructions define the robot, permitted observations and action conventions; developer instructions specify workspace use and interaction rules. Below we reproduce six examples from the evaluated episodes, grouped by embodiment. The task goal and current observations accompany these instructions, while Appendix B describes the callable tools. Quotes are verbatim excerpts with whitespace normalised; separate paragraphs omit intervening text. Environment-side code control is disabled throughout the experiments.

C.1 Tabletop manipulation

Both tabletop interfaces expose RGB images and robot state, but their Cartesian targets refer to different physical points. RoboDojo commands the grasp point between the jaws, whereas RoboLab commands the gripper flange. Their gripper conventions also differ: zero closes in RoboDojo and opens in RoboLab. These distinctions are stated explicitly in the instructions.

RoboDojo / grasp-point control

System instructions (excerpts)

“You are controlling a real robot embodiment named ‘robodojo-*arx-x5*’. You receive RGB camera images, the current world-frame grasp-point state of both arms in the same 14 dimensions *move_eef* takes, arm joint angles as context you cannot command, and a task instruction. Move with *move_eef* by naming only the world-frame dimensions you want to change. Cartesian targets must be estimated from RGB; no depth or world-coordinate query is available. Respond with exactly one tool call per turn. After each motion the next observation reports how far the grasp point ended up from what you asked for, so check it before assuming a motion landed.”

“Gripper 0 is fully closed, 1 is fully open, and commanding 0 always closes as far as the object allows.”

Developer instructions (excerpt)

“Use *move_eef* for robot motion. Each result includes fresh observations. Never use shell, files, web or other tools to control the scene or read hidden state. Execute one motion at a time and inspect its result.”

RoboLab / flange-pose control

System instructions (excerpts)

“You control the fixed-base DROID robot in RoboLab. Complete the exact upstream task instruction using supplied RGB and measured proprioception only. The robot has one Franka arm and a Robotiq gripper; it has no mobile base, torso or head control. All positions are in metres relative to the robot articulation root; all quaternions are WXYZ. *ee_pos/ee_quat* describe the gripper base_link flange, not the fingertips. *eef_pos/eef_quat* describe a differently oriented reporting frame, NOT the IK target frame.”

“*gripper_close=0* opens and 1 closes. Use *move_robot* to combine arm and gripper in the same native action; closure begins alongside movement, not on arrival. Separate tools are sequential. A tool segment costs steps once; the native rate is 15 Hz.”

Developer instructions

“Shell/code and /workspace are for local calculations and notes. Read-only /observations contains only allowed RGB and proprioception. All movement and fresh observations must use the robot tools. Never read or change simulator/evaluator files or seek external task solutions.”

C.2 Mobile manipulation

Mobile manipulation adds coordination between the arm, gripper, torso and base. RoboCasa uses incremental end-effector commands; BEHAVIOR-1K accepts absolute arm poses and physical base velocities. The instructions explain which components move together and which hold their current targets.

RoboCasa / incremental mobile manipulation

System instructions (excerpts)

“You control the standard RoboCasa365 PandaOmron robot using benchmark-specific robot tools. Complete the exact task instruction from the environment. Use only supplied RGB and proprioception. There is no depth sensor enabled in this standard evaluation. EEf pose observations are relative to the robot base; quaternions are XYZW. Actions are the native normalized 12D controller inputs, NOT the absolute IK targets of other benchmarks. `move_eef` supplies base-frame OSC increments: normalized translation scales to up to 0.05m and rotation to 0.5rad per step. Repeating a delta repeats motion; use small magnitudes and short segments near objects. Base and torso commands are normalized, not metres or m/s. Gripper `close=1/open=0` persists. Unspecified motion inputs are zero. `control_mode=0` uses achieved arm pose; 1 follows desired pose when base moves.”

Developer instructions

“Use shell/code only for local computation and memory in `/workspace`. Read-only `/observations` contains RGB PNG and measured robot state. All motion and new observations must use robot tools. Do not access simulator/evaluator files or bypass the interface. A final answer does not terminate the episode.”

BEHAVIOR-1K / coordinated dual-arm motion

System instructions (excerpts)

“You control an R1Pro mobile dual-arm robot through benchmark-specific `move_robot`, `move_base`, `move_arms`, `set_grippers` and (when available) `move_torso` tools. Complete the named household task. Use `move_robot` to combine both arm poses, base velocities, gripper openings and available trunk targets in one simultaneous bounded action. Each control step applies all components together and counts once toward the budget; separate tool calls execute sequentially, not concurrently. Combined gripper changes begin alongside motion, not after arm arrival.”

“Use only onboard RGB, depth and proprioception. EEf positions and XYZW quaternions are relative to the robot articulation root, not world coordinates. Read the measured starting poses. Base local `+x` forward, `+y` left, `+z` up; base velocities in m/s and yaw rad/s. Unspecified arms hold their observed pose during the action; gripper targets persist (0 closed, 1 open). Base velocities default to zero on each call.”

Developer instructions (excerpt)

“Use shell, code execution and files in `/workspace` for computation, planning, maps and persistent memory. Allowed sensor exports are read-only in `/observations`; each numbered directory includes RGB PNGs, raw depth NPYs and proprioception.”

C.3 Driving and flight

Driving and flight require different observation and control descriptions. The WheeledLab example supplies onboard visual and proprioceptive observations with speed and steering commands. The OmniDrones example instead supplies native actor measurements and direct rotor inputs. Both distinguish policy observations from external review-camera views.

WheeledLab / onboard driving

System instructions (excerpts)

“You drive a real-collision simulated MuSHR in a RobotWorld custom driving course. Observation profile robotworld-onboard-v2: front camera, ideal wheel/steering encoders and body IMU gyro/tilt. These simulate sensors a real car can carry; no added sensor noise yet. No global position/yaw, true translational velocity/slip, full map, obstacle coordinates, checkpoint progress, next waypoint, gate pose, gate clearance Boolean, friction map or future perturbations are given. Infer road shape, hazards and stopping locations from the onboard image. Review-camera video and independent evaluator records are NOT policy inputs. Do not try to recover private state through shell/files/network.”

“Only observe and drive are enabled; no per-tick program execution tool is available.”

Developer instructions

“Use shell only for calculations in /workspace. Only dynamic tools control physics. /observations contains allowed policy observations. A final answer does not end the episode. Non-action interaction budget: 15 consecutive or 30 total calls/empty turns. Auxiliary tool calls and robot calls executing zero control steps each cost one. A normal turn with no executed control steps and no counted calls costs one. Confirmed control steps reset only the consecutive counter. API retries do not consume this budget. The evaluator ends the episode at either limit.”

OmniDrones / direct rotor control

System instructions (excerpts)

“You directly control a Hummingbird quadrotor through FOUR normalized rotor commands in native order. action_transform=null; no Lee controller, hover policy, automatic attitude stabilizer or trajectory tracker runs for you. All four commands are simultaneous, each in [-1,1]: -1 targets zero thrust, +1 maximum thrust. Zero is half maximum steady-state thrust, NOT a neutral/hover command. Native actuator first-order throttle response remains active.”

“Only the original actor policy group is supplied. It may include simulator-derived state, commands or a task’s native predictor; consult this task’s explicit observation contract. Never claim pure visual control if state is included. Critic/privileged evaluator observations are excluded. Third-person review video is never a policy input. Native sensor images, if any, are labelled separately.”

Developer instructions

“Only dynamic robot tools control physics. Shell is for calculations. Continue until the episode terminates or budget ends. A final answer does not end the episode. Non-action interaction budget: 15 consecutive or 120 total calls/empty turns. Auxiliary tool calls and robot calls executing zero control steps each cost one. A normal turn with no executed control steps and no counted calls costs one. Confirmed control steps reset only the consecutive counter. API retries do not consume this budget. The evaluator ends the episode at either limit.”

D Full Quantitative Results

Section 5 presents the aggregate results and domain comparisons. Here we provide the complete task-level outcomes and numerical resource distributions for all five models.

D.1 All task–model outcomes

Table 11 includes every task exactly once. Columns A, O, K, D and G denote Astra, Opus 5.5, Kimi K3, DeepSeek V4.1 Flash and Gemini 3.8 Flash. A green check denotes retained success; a red cross denotes retained failure. A dagger marks an adjudicated record. The horizon is the task’s recorded control-step limit; N and W indicate native and World-state scoring profiles. Task domains and descriptions are provided in Appendix A.

Table 11 Complete 84-task inventory and five-model outcomes.

ID	Source / task identifier	Limit	Score	A	O	K	D	G
RoboDojo								
01	hang_mugs	800	N	×	×	×	×	×
02	sweep_blocks	1000	N	×	✓	×	×	×
03	pour_liquid_into_cup	400	N	×	×	×	×	×
04	make_toast	1400	N	×	×	×	×	×
05	store_laptop_and_headphones	800	N	×	×	×	×	×
06	insert_tubes	500	N	×	×	×	×	×
07	plug_in_charger	400	N	✓	×	×	×	×
08	pour_balls_into_vase	600	N	×	×	×	×	×
09	play_Xylophone	500	N	×	×	×	×	×
10	fill_pen_holder	1100	N	×	×	×	×	×
11	fill_egg_holder	700	N	×	×	×	×	×
12	make_kong	600	N	×	×	×	×	×
13	pour_by_language	800	N	×	×	×	×	×
14	match_and_pick_from_conveyor	700	N	✓	✓	×	×	✓
15	deposit_coin	300	N	×	×	×	×	×
BEHAVIOR-1K								
16	carrying_in_groceries	2000	N	×	×	×	×	×
17	clean_up_your_desk	2000	N	×	×	×	×	×
18	slicing_vegetables	2000	N	×	×	×	×	×
19	sorting_vegetables	2000	N	×	×	×	×	×
20	clean_boxing_gloves	2000	N	×	×	×	×	×
21	putting_up_Christmas_decorations_inside	2000	N	×	×	×	×	×
22	setting_the_table	2000	N	×	×	×	×	×
23	putting_dishes_away_after_cleaning	2000	N	×	×	×	×	×
24	can_meat	2000	N	×	×	×	×	×
25	freeze_pies	2000	N	×	×	×	×	×
RoboCasa								
26	CountertopCleanup	600	N	×	×	×	×	×
27	SortingCleanup	3000	N	×	×	×	×	×
28	CoffeeSetupMug	600	N	✓	×	×	×	×
29	CloseDrawer	450	N	✓	×	✓	×	×
30	NavigateKitchen	450	N	✓	×	×	×	×
31	PackIdenticalLunches	3900	N	×	×	×	×	×
32	OrganizeMugsByHandle	1350	N	×	×	×	×	×
33	LoadDishwasher	1800	N	✓	×	×	×	×
34	MicrowaveCorrectMeal	1500	N	×	×	×	×	×
35	ResetCabinetDoors	3300	N	×	×	×	×	×

Continued on next page

Table 11 continued

ID	Source / task identifier	Limit	Score	A	O	K	D	G
RoboLab								
36	ToolOrganizationTask	2700	N	✗	✗ [†]	✗	✗	✗
37	NonHammerToolsInRightBinTask	2700	N	✗	✗ [†]	✗	✗	✗
38	FoodPacking2CansTask	2700	N	✓	✗ [†]	✗	✗	✗
39	RubiksCubeLeftOfBowlTask	450	N	✓	✓	✗	✗	✗
40	FruitsOnPlate3Task	3000	N	✓	✓	✗	✗	✗
41	PutTwoMugsOnShelfTask	2700	N	✗	✗ [†]	✗	✗	✗
42	BlockStackingSpecifiedOrderTask	1350	N	✓	✗ [†]	✗	✗	✗
43	ClutterPlasticTask	2700	N	✓	✓	✗	✗	✗
44	ReorientWhiteMugsTask	900	N	✗	✗	✗	✗	✗
45	WhiteMugInCenterOfTableTask	450	N	✓	✓	✗	✗	✗
HumanoidSoccer								
46	play-soccer	300	N	✗	✗	✗	✗	✗
AI-CPS								
47	22	300	N	✗	✗	✗	✗	✗
48	23	300	N	✗	✗	✗	✗	✗
49	24	300	N	✓	✗	✗	✗	✗
WheeledLab								
50	mushr-drift	400	W	✗	✗ [†]	✗	✗	✗
51	fltentht-drift	400	W	✗	✗	✗	✗	✗
52	elevation	200	N	✗	✗	✗	✗	✗
53	visual	150	W	✗	✗ [†]	✗	✗	✗
54	rw-courtyard	2000	N	✓	✓	✓	✗	✗
55	rw-hairpins	2000	N	✓	✓	✗	✗	✗
56	rw-gate-dock	2000	N	✗	✗	✗	✗	✗
57	rw-drift-switch	2000	N	✗	✗	✗	✗	✗
58	rw-twin-beam	2000	N	✗	✗	✗	✗	✗
59	rw-reverse-bay	2000	N	✗	✗	✗	✗	✗
60	rw-parallel-park	2000	N	✗	✗	✗	✗	✗
Bench2Dex								
61	41	681	N	✗	✗	✗	✗	✗
62	42	986	N	✗	✗	✗	✗	✗
63	43	1149	N	✗	✗	✗	✗	✗
64	44	1080	N	✗	✗	✗	✗	✗
65	45	964	N	✗	✗	✗	✗	✗
66	46	1061	N	✗	✗	✗	✗	✗
67	47	593	N	✗	✗	✗	✗	✗
68	48	1213	N	✗	✗	✗	✗	✗
69	49	1082	N	✗	✗	✗	✗	✗
Digit								
70	digit_walk_hand_tracking	700	W	✗	✗	✗	✗	✗
Flamingo								
71	wheel_legged_jump_balance	1000	W	✗	✗	✗	✗	✗
Go2 Push								
72	quadruped_push_recovery	1000	W	✗	✓	✗	✗	✗
OmniDrones								
73	drone_payload_hover	500	W	✗	✓	✗	✗	✗
74	drone_inverted_pendulum_tracking	620	W	✗	✗	✗	✗	✗
OmniIsaacGymEnvs								
75	anymal_rough_terrain	800	W	✗	✗	✗	✗	✗
ReflexBench								
76	cup_ball_catching	100	N	✗	✓	✗	✗	✗

Continued on next page

Table 11 continued

ID	Source / task identifier	Limit	Score	A	O	K	D	G
Robot Lab								
77	a1_front_leg_handstand	500	W	✗	✗	✗	✗	✗
SteadyTray								
78	tray_balancing_walk	1000	W	✗	✗	✗	✗	✗
TTRL								
79	humanoid_table_tennis_return	1500	W	✗	✗	✗	✗	✗
VolleyBots								
80	drone_volleyball_1v1	1000	N	✓	✓	✗	✓	✗
81	drone_volleyball_solo_juggle	800	W	✗	✓	✗	✗	✗
Wheel-Legged								
82	wheel_legged_upright_recovery	2000	W	✗	✗	✗	✗	✗
83	wheel_legged_rough_terrain	2000	W	✗	✗	✗	✗	✗
Wheeled Quadruped								
84	rear_wheel_upright_balance	1000	W	✗	✗	✗	✗	✗

D.2 Resource distributions

Tables 12 and 13 report medians and interquartile ranges (IQRs) across tasks. Time and call counts cover all 84 episodes per model, including failures. Intervals describe task variation, not repeat-trial uncertainty. Full logs can extend beyond the evaluation boundary; timing includes setup, waiting and execution.

Table 12 Time and interaction use. Medians and interquartile ranges across tasks.

Model	Elapsed time (min)		Robot requests		All tool calls	
	Median	IQR	Median	IQR	Median	IQR
Astra	30.9	[15.2, 61.2]	58	[25.5, 94]	70	[30, 138.8]
Opus 5.5	54	[24.5, 137.3]	37	[16, 87.2]	57.5	[31, 148]
Kimi K3	67.6	[21.3, 181.7]	42.5	[11.8, 70]	61.5	[18.5, 95]
DeepSeek	16	[11.1, 40.5]	22	[5, 59.2]	33	[18, 70]
Gemini	12.8	[10, 25.8]	11	[0, 36.8]	31	[15, 52.2]

Control-step statistics use the available observed step counts from full logs: 84 episodes for Astra, 80 for Opus 5.5, 82 for Kimi, 75 for DeepSeek and 60 for Gemini. Missing entries are excluded. Images supplied within robot feedback are not additional image-view calls; zero explicit views therefore does not imply absence of visual observations.

Table 13 Physical execution and auxiliary-tool use. Medians and interquartile ranges across tasks.

Model	Control steps		Shell calls		Image views	
	Median	IQR	Median	IQR	Median	IQR
Astra	627	[298, 1165]	4.5	[0, 21.2]	0	[0, 0]
Opus 5.5	458	[178.5, 913.8]	15	[0, 28]	0	[0, 10]
Kimi K3	449.5	[158, 918.5]	1	[0, 14.2]	0	[0, 1.2]
DeepSeek	347	[62, 742]	8.5	[0, 16]	2.5	[0, 8]
Gemini	315	[78.5, 625]	15	[9.8, 23]	0	[0, 2.2]

D.3 Aggregate token and cost accounting

The values in Table 14 accompany the three point plots in Figure 11. All three measures reproduce the project website’s valid per-task mean scaled to 84 tasks, as retrieved on 7 October 2026. Missing or zero-flagged usage is excluded from the mean, not counted as zero. Kimi has 57 valid token and cost records; each other model has 84. Thus Kimi’s aggregate consumption is an estimate extrapolated to the full task set, not the sum of the recorded values. Tokens include cached input and output and do not sum repeated cumulative updates. Monetary values use the website’s list-price estimates rather than invoices. Elapsed time uses all 84 website-reported durations per model, whose rounding can cause small differences from raw timestamps. All measures include failed tasks; full-run consumption can include execution after a retrospective scoring boundary. Provider token definitions, cache accounting and unit prices differ, so token and cost rankings need not coincide.

Table 14 Aggregate consumption for the five-model evaluation. Monetary values are reported list-price estimates. Kimi token and cost totals extrapolate from 57 valid records to 84 tasks.

Model	Tokens	Estimated USD	Elapsed hours
Astra	941M	9,912.80	54.6
Opus 5.5	1.63B	2,916.43	140.3
Kimi K3	2.00B	624.09	163.8
DeepSeek V4.1 Flash	907M	34.49	35.7
Gemini 3.8 Flash	303M	66.68	28.7

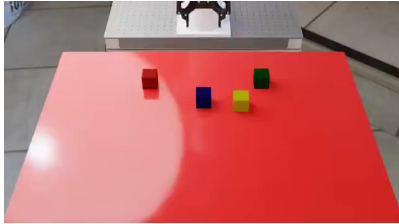
E Successful Trajectories

We present four deliberately selected successes covering sequential manipulation, dynamic stabilisation, recovery after a rejected action, and competitive interaction. Each case connects observed state, the agent’s next decision, and the resulting execution feedback. Every image and quoted excerpt comes from our own retained archive. Selection is qualitative; these examples do not estimate behavioural prevalence.

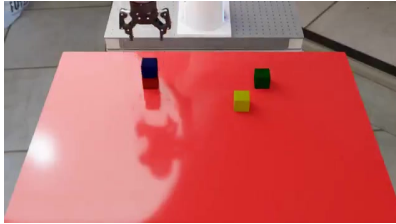
Event numbers are zero-based indices in the merged trace. “Returned step” is the tool event’s recorded step, not a guarantee that every requested step executed. Tool notes express the agent’s stated intent and can be wrong. Evaluator fields establish the retained outcome; images and visible messages explain the sequence.

E.1 Astra: building and checking an ordered stack

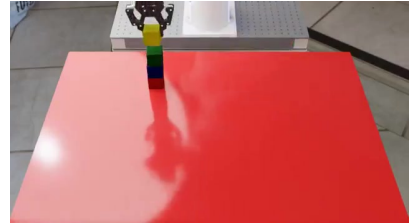
Record: astra-62. **Outcome:** success. **Robot requests:** 35. **Scored steps:** 635/1350.



Video 0.10 s



Video 16.90 s



Video 40.98 s

Figure 20 Astra stacking. Recorded external-camera frames from **astra-62**. The final shown frame still precedes the terminal release request. The complete run, rather than this frame alone, supplies the success label.

Stage 1. The agent leaves the red block as the base and explicitly orders the remaining placements. Three initial movements probe the image-to-robot relation before descent.

astra-62 | event 4

Agent message: “I’ll leave red as the base, then place blue, green, and yellow on top, checking each grasp and placement. ”

Stage 2. After placing blue, the agent requests a vertical retreat before starting the next grasp. This separates release verification from moving toward green; the transcript then reuses the established stack centre.

astra-62 | event 276 | returned step 270

Agent tool note: “Retract vertically to verify the released blue-on-red stack and clear it before approaching green.”

Stage 3. The same approach–grasp–lift–place sequence is repeated for green and yellow. The final request opens the gripper; the retained evaluator reports success at step 635. Because the final client receipt is missing, we preserve the last completed-event step of 634 separately.

astra-62 | event 684

Agent tool note: “Release yellow onto green without moving the arm.”

E.2 Opus 5.5: repeated contacts under rotor-level control

Record: task-158. **Outcome:** success. **Robot requests:** 274. **Scored steps:** 800/800.

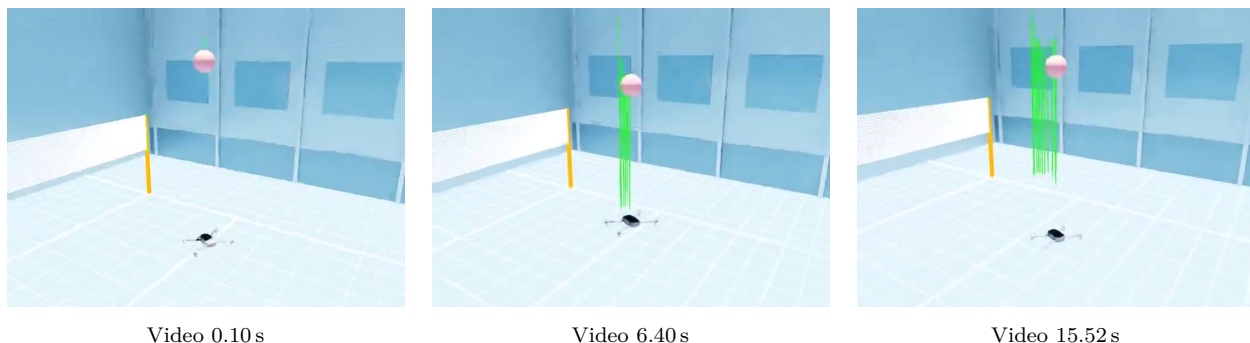


Figure 21 Opus 5.5 juggling. Reviewer-camera frames from **task-158**. The policy receives a native actor vector, not these rendered views. Hit counts and final-window validity come from the retained checker.

Stage 1. The first requests use two-step segments to level the vehicle and revise the rotor mixing. A native motor mapping and lag mediate the requested inputs, so command values alone do not establish achieved thrust.

task-158 | event 12 | returned step 6

Agent tool note: “corrected mixer (roll arm .21, pitch arm .13, equal trim)”

Stage 2. The later trace alternates descent, upward acceleration and post-contact cuts. The note below identifies a planned strike, while the independent event counters establish whether contact and height qualification actually occurred.

task-158 | event 599 | returned step 437

Agent tool note: “strike 9 accel, tilt trim”

Stage 3. The agent continues until step 800. The retained record reports 15 true hits and 14 height-qualified hits, exceeding the required four, with the final 0.6-second airborne condition satisfied and no recorded overriding failure. The 274 requests illustrate frequent feedback in this particular success.

task-158 | event 1070 | returned step 800

Agent tool note: “hover to episode end”

E.3 Gemini: recovering from an unreachable lift

Record: task-272. **Outcome:** success. **Robot requests:** 48. **Scored steps:** 511/700.

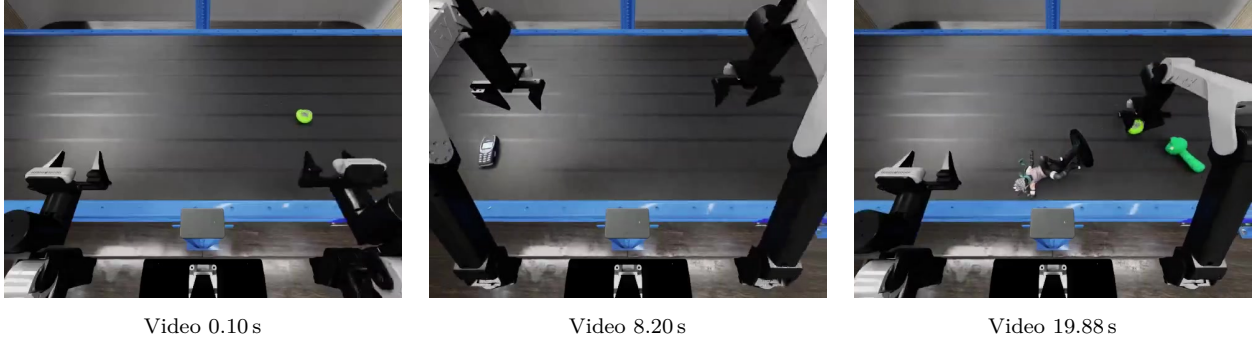


Figure 22 Gemini conveyor matching. Recorded head-camera frames from **task-272**. The decisive recovery is resolved by consecutive tool receipts at events 550–559, rather than inferred solely from sampled frames.

Stage 1. The agent tracks a matching green object among distractors and attempts a grasp. The target is moving, and robot motions also advance the conveyor. The following request is the later right-arm grasp at the observed travel lane.

task-272 | event 537 | returned step 501

Agent tool note: “Descending right arm to grasp height $z = 0.7725$ at $(0.30, -0.02)$ over the green Tamagotchi and closing the gripper upon arrival.”

Stage 2. A subsequent lift to 0.95 m is rejected without increasing the recorded step beyond 501. The agent lowers the requested height to 0.85 m; the next receipt confirms progress to step 508. This is a concrete revision after negative execution feedback.

task-272 | event 550 | returned step 501

Tool feedback: “right arm pose unreachable (planner status: Fail).” [excerpt]

task-272 | event 551 | returned step 508

Agent tool note: “Lifting right arm with grasped Tamagotchi to $z = 0.85$, which is a verified reachable intermediate height.”

Stage 3. The final request retracts toward home while maintaining the grasp. The episode ends at step 511 with retained success. The requested motion duration and the actual terminal step differ because evaluation can stop an in-flight segment.

E.4 DeepSeek: correction, interception and a terminal win

Record: task-350. **Outcome:** success. **Robot requests:** 39. **Scored steps:** 241/1000.

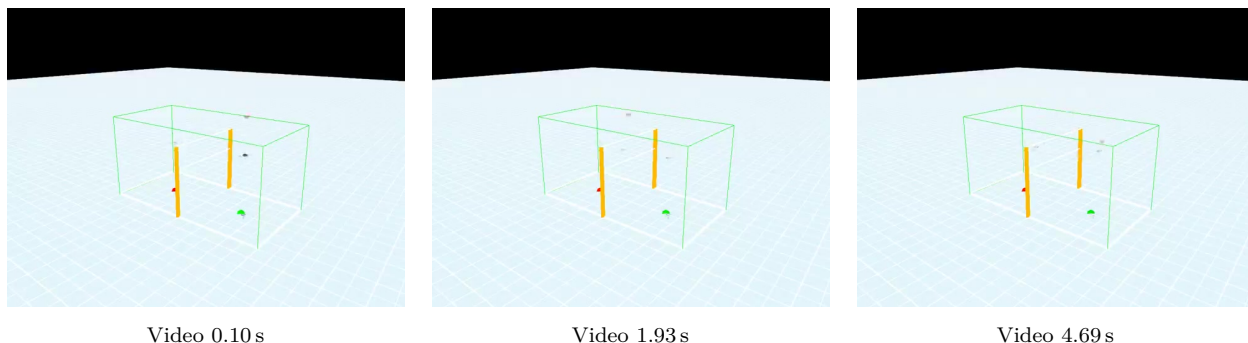


Figure 23 DeepSeek volleyball 1v1. Reviewer video from task-350. The result is the native terminal win for the controlled actor; surviving or touching the ball alone would not suffice.

Stage 1. After an initial collective command, the agent explicitly revises its hover calculation in response to a larger-than-expected rise. Its explanation is an observable message; it is not treated as a direct measurement.

task-350 | event 17

Agent message: “The vehicle rose faster than expected” [excerpt]

Stage 2. The transcript records an attempted return followed by recovery away from the net. The later message below identifies the next incoming ball and a new interception target. These messages document the intended strategy, while the state trace records actual motion.

task-350 | event 199

Agent message: “The opponent has returned the ball — it’s now heading to my side (arriving $\sim x=1.24$, $z=2.0$ around step 192). Repositioning to intercept.”

Stage 3. The native result records four total contacts, two by each side, and a win for the controlled actor. The retained success occurs at step 241 of a 1,000-step horizon. This is one winning episode, not an estimate of match win probability.

F Observable Behavioural Patterns

The cases support the following descriptive categories. Labels attach to linked local sequences of observations, agent outputs, tool requests and receipts. One episode may exhibit several categories. We do not turn this purposive case selection into a corpus-wide frequency estimate or claim inter-annotator agreement that was not measured.

Pattern	Trace evidence	Interpretation boundary
Sequential decomposition	Astra stacking names the placement order and reuses a stack centre after release/retraction.	An explicit plan is evidence of intent; the final checker confirms the completed arrangement.
Feedback-driven correction	Gemini revises a rejected 0.95 m lift to 0.85 m; the next receipt advances from step 501 to 508.	The consecutive request/receipt pair supports a local correction, not a causal effect of a general recovery policy.
Frequent dynamic feedback	Opus 5.5 juggling uses 274 robot requests over 800 control steps with repeated strike/descent phases.	This successful case does not establish that more calls universally improve success.
Calibration revision	DeepSeek changes its hover computation after reporting excessive rise.	A visible self-correction is distinct from independently verified correctness of its entire controller.
Contact/grasp reassessment	Kimi pouring repeatedly reports missed closure and changes approach or wrist orientation.	The agent notices difficulty; final failure should not be described as an unobserved or hallucinated success.
Pre-action analysis saturation	DeepSeek computer-use episode reaches a non-action boundary at step zero.	No robot motion is available for diagnosing physical control ability in that episode.
Post-boundary continuation	Opus 5.5 stacking has a historical success after an earlier scored interaction cutoff.	Late footage cannot be counted as success under the retained budget.

F.1 What counts as using feedback

Receiving a fresh observation, explicitly viewing an image file, and changing an action after feedback are different events. The present archive directly counts tool and image-view events; deciding whether visual information changed an action requires a local evidence chain. For instance, the Gemini case links an unreachable-pose response to a smaller requested lift and subsequent confirmed execution. A low image-view count alone does not show that an agent ignored images already supplied in robot feedback.

F.2 How to read agent claims

Tool notes and messages record the agent’s intentions or interpretation; they do not independently prove retention, contact or completion. In Kimi pouring, the notes acknowledge a tipped bottle and missed grasps, so the failure should not be described as a false claim of final success. In Astra stacking, the native result reports success although the final release request lacks terminal client feedback. We preserve that distinction between missing feedback and failed execution.

G Matched-Task Comparisons

The current five-model snapshot supports matched-task observational comparisons, not controlled ablations of history, code control or reasoning budget. All five models encounter the same named 84-task inventory with matching per-task seed metadata. Differences between providers, collection times and incomplete build metadata remain possible confounders. We therefore report outcome overlap and costs for matched successful tasks without attributing them to an isolated interaction strategy.

G.1 Success overlap

Each off-diagonal cell below counts tasks solved by both models; diagonal cells are each model’s success count. Across the five sets, 21 unique tasks are solved by at least one model. The matrix demonstrates overlap without assuming independent model errors or an executable model-selection oracle.

Model	Astra	Opus 5.5	Kimi	DeepSeek	Gemini
Astra	16	8	2	1	1
Opus 5.5	8	13	1	1	1
Kimi	2	1	2	0	0
DeepSeek	1	1	0	1	0
Gemini	1	1	0	0	1

G.2 Astra and Opus 5.5 on shared successes

Eight tasks are successful for both Astra and Opus 5.5 in this retained snapshot. Table 16 separates robot requests, all recorded calls, and retained result steps. These are full retained successful episodes; they are not a common intermediate-state alignment or a time-to-first-success experiment. The resource fields are audited from each task’s own event stream.

Table 16 Matched successful tasks. Every numeric pair is Astra / Opus 5.5.

Task	Robot requests	All calls	Result steps
Conveyor matching	45 / 35	45 / 35	380 / 347
Plastic clutter sorting	41 / 104	46 / 107	784 / 1497
Three fruits on a plate	21 / 31	22 / 58	284 / 419
Cube left of bowl	17 / 18	24 / 31	286 / 239
White mug centring	18 / 22	20 / 42	280 / 341
Volleyball 1v1	30 / 152	30 / 157	135 / 498
Courtyard driving	16 / 37	16 / 42	703 / 1006
Hairpin driving	33 / 88	33 / 88	1383 / 1829

Fewer tool requests need not imply fewer physical steps: a request can hold an action for a longer segment, and auxiliary analysis changes the total-call count without advancing physics. Likewise, a shorter recorded elapsed time can reflect earlier failure. Controlled claims about history retention, segment length or coding control require paired reruns in which those variables are explicitly changed while the task, state, budget and checker are held fixed; this archive does not supply such an experiment.

H Failure Cases and Outcome Adjudication

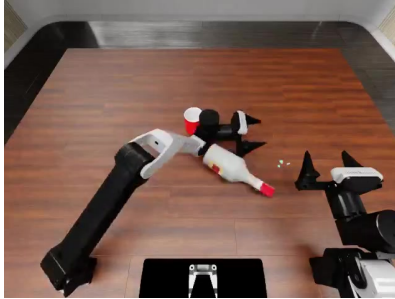
We distinguish failure after physical interaction, exhaustion before robot execution, and a mismatch between historical terminal results and the scored budget boundary. These examples are selected to clarify the evidence chain; they are not an exhaustive taxonomy or a measured distribution of failure causes.

H.1 Kimi: repeated grasp revision without a completed pour

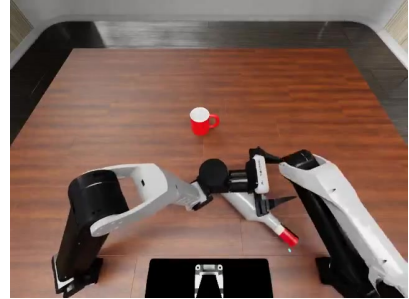
Record: task-25. **Outcome:** failure. **Robot requests:** 27. **Scored steps:** 400/400.



Video 0.10 s



Video 6.40 s



Video 15.52 s

Figure 24 Kimi pouring failure. Head-camera frames from task-25. The bottle is upright initially and later lies on the table. Agent notes acknowledge failed acquisition; the retained episode ends unsuccessfully at step 400.

Stage 1. The initial approach advances 42 steps. The following tool note reports that contact knocked the bottle onto its side, then requests closure. This is evidence that the agent noticed the changed object state.

task-25 | event 47 | returned step 48

Agent tool note: “The approach knocked the bottle onto its side” [excerpt]

Stage 2. The next revision acknowledges that closure missed. The agent subsequently rotates the wrist, raises for clearance, shifts laterally and retries. These actions consume time without establishing a stable bottle grasp.

task-25 | event 54 | returned step 72

Agent tool note: “The first closure missed because the gripper was above the fallen bottle.” [excerpt]

Stage 3. Control later switches to the right arm, but the final request still repositions above the bottle. The record ends at the 400-step horizon with failure. The trace supports unsuccessful grasp acquisition and budget exhaustion; it does not support a claim that the model declared the pour complete.

task-25 | event 327 | returned step 400

Agent tool note: “The right gripper is now beyond the bottle mouth” [excerpt]

H.2 DeepSeek: analysis consumes the budget before any action

Record: task-232. **Outcome:** failure. **Robot requests:** 0. **Scored steps:** 0/593.

The Bench2Dex computer-use record contains no robot-tool request and no confirmed physical step. Its terminal non-action ledger reaches 15 consecutive auxiliary units. The visible merged trace contains ten shell executions and four explicit image views; the ledger total is 15 and flags incomplete accounting. We report both quantities rather than inventing a missing fifteenth invocation.

Stage 1. The agent constructs forward-kinematics scripts and inspects supplied observations. These are permitted workspace activities, but they do not advance the robot.

task-232 | event 4

Shell command excerpt (Python source):

```
def T(xyz, rpy):  
    M=np.eye(4)  
    M[:3,:3]=rpy2R(*rpy)  
    M[:3,3]=xyz  
    return M
```

Stage 2. Subsequent calls crop and inspect images of the hands, keyboard and mouse, and revise the kinematic calculation. This documents preparatory analysis, with no recorded attempt to actuate the robot.

task-232 | visible auxiliary events | reviewer summary

Shell executions: 10. **Explicit image views:** 4.

Robot requests: 0. **Confirmed control steps:** 0.

Terminal ledger: total 15; consecutive peak 15; stop reason `nonaction_consecutive_limit`.

Stage 3. The run is unsuccessful under the retained interaction protocol. Since there is no executed robot action, this case cannot establish whether a proposed joint command would have succeeded or failed physically. It identifies a failure to convert available analysis into execution before the interaction limit.

H.3 Attribution limits

A malformed request, a planner rejection, a dropped host event, a simulator failure and an unmet task predicate are different mechanisms. The scored label alone does not identify which one occurred. In particular, the discrepancy between 14 visible auxiliary events and 15 charged units is an accounting limitation; it is not evidence of an unlogged physical action. The raw response, converted request, validation receipt and environment state must be traced before assigning a more specific cause.

H.4 Opus 5.5: late success beyond the scored boundary

Record: task-82. **Outcome:** failure. **Robot requests:** 40. **Scored steps:** 313/1350.

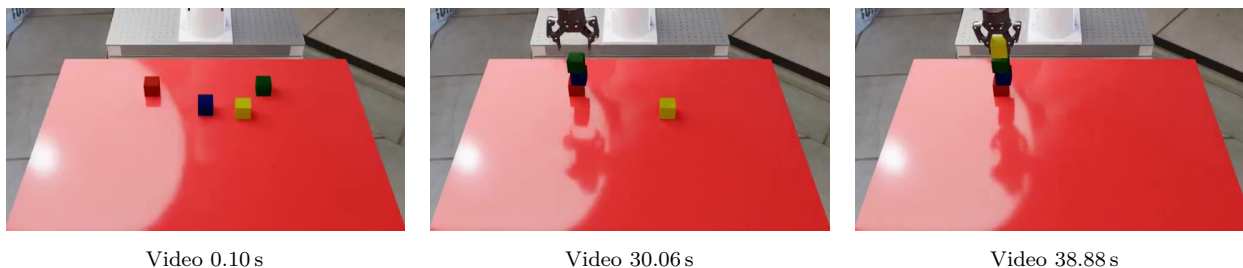


Figure 25 Opus 5.5 stacking: historical continuation. Frames from the full retained video of **task-82**. The later two views lie beyond the scored step-313 budget boundary; they illustrate continued work, not valid success within that budget.

Stage 1. The agent calibrates motion, grasps blue and releases it onto red. Its messages describe a stable partial stack. The retrospective non-action ledger reaches the total limit of 30 at control step 313.

task-82 | event 325

Agent message: “Blue is now on red, slightly offset but stable. Next I’ll locate the green block.”

Stage 2. The historical log continues: the agent grasps green, places it, and proceeds to yellow. These later events are available for diagnosing strategy, but are outside the retained scoring window.

task-82 | event 527

Agent message: “Green is released on blue, and the side view shows red–blue–green stacked. Lifting straight up, then going for yellow, which I estimated at about (0.478, 0.109).”

Stage 3. The original run result preserves success after 602 executed control steps, while completed merged environment events reach step 601. The retained manifest instead reports failure at step 313 with reason `nonaction_total_limit`. Its scoring source explicitly identifies retrospective budget adjudication.

task-82 | two outcome records, different evaluation boundaries

Retained score: failure at step 313; total non-action limit 30.

Historical run: success; 602 executed steps.

Merged execution stream: completed events through step 601.

Interpretation: the late native success does not change the scored budget-limited outcome.

H.5 Complete adjudication ledger

Twenty-one retained records are marked adjudicated: 15 Opus 5.5 and 6 Kimi records. Nine lack a Boolean success value in the original run record and are assigned retained failure; twelve preserve a Boolean historical outcome. Of the latter, three historical successes become failures at an earlier budget boundary. The ledger below records all affected tasks, including adjudications that leave a historical failure unchanged. A missing original Boolean is shown as “unavailable”, not silently converted to a native failure.

Trace	Model / task	Original	Scored step	Observed step
task-1	Kimi K3 / CountertopCleanup	failure	489	600
task-104	Opus 5.5 / visual	failure	19	143
task-13	Kimi K3 / OrganizeMugsByHandle	unavailable	335	335
task-17	Kimi K3 / MicrowaveCorrectMeal	unavailable	372	372
task-19	Kimi K3 / ResetCabinetDoors	unavailable	2420	2420
task-2	Opus 5.5 / SortingCleanup	unavailable	778	778
task-3	Kimi K3 / SortingCleanup	unavailable	306	306
task-5	Kimi K3 / CoffeeSetupMug	failure	192	600
task-50	Opus 5.5 / carrying_in_groceries	failure	265	1112
task-52	Opus 5.5 / clean_up_your_desk	failure	290	2001
task-54	Opus 5.5 / slicing_vegetables	failure	240	2001
task-58	Opus 5.5 / clean_boxing_gloves	failure	220	2001
task-60	Opus 5.5 / putting_up_Christmas_decorations_inside	unavailable	2000	2000
task-62	Opus 5.5 / setting_the_table	unavailable	1114	1114
task-64	Opus 5.5 / putting_dishes_away_after_cleaning	failure	787	1132
task-70	Opus 5.5 / ToolOrganizationTask	unavailable	1025	1025
task-72	Opus 5.5 / NonHammerToolsInRightBinTask	unavailable	1392	1392
task-74	Opus 5.5 / FoodPacking2CansTask	success	425	531
task-80	Opus 5.5 / PutTwoMugsOnShelfTask	success	174	466
task-82	Opus 5.5 / BlockStackingSpecifiedOrderTask	success	313	601
task-98	Opus 5.5 / mushr-drift	failure	120	205

All retained outcomes in this ledger are failures. The supplementary inventory preserves the page hashes and both outcome fields. Some entries have incomplete invocation accounting or lack terminal client feedback; those limitations remain attached to the records. The current local task implementation is not substituted for the historical prompt or checker contract when interpreting these runs.